

Deep learning
Master's Degree in Artificial Intelligence
2025/26

Unit 5C – GANs (Generative Adversarial Networks)

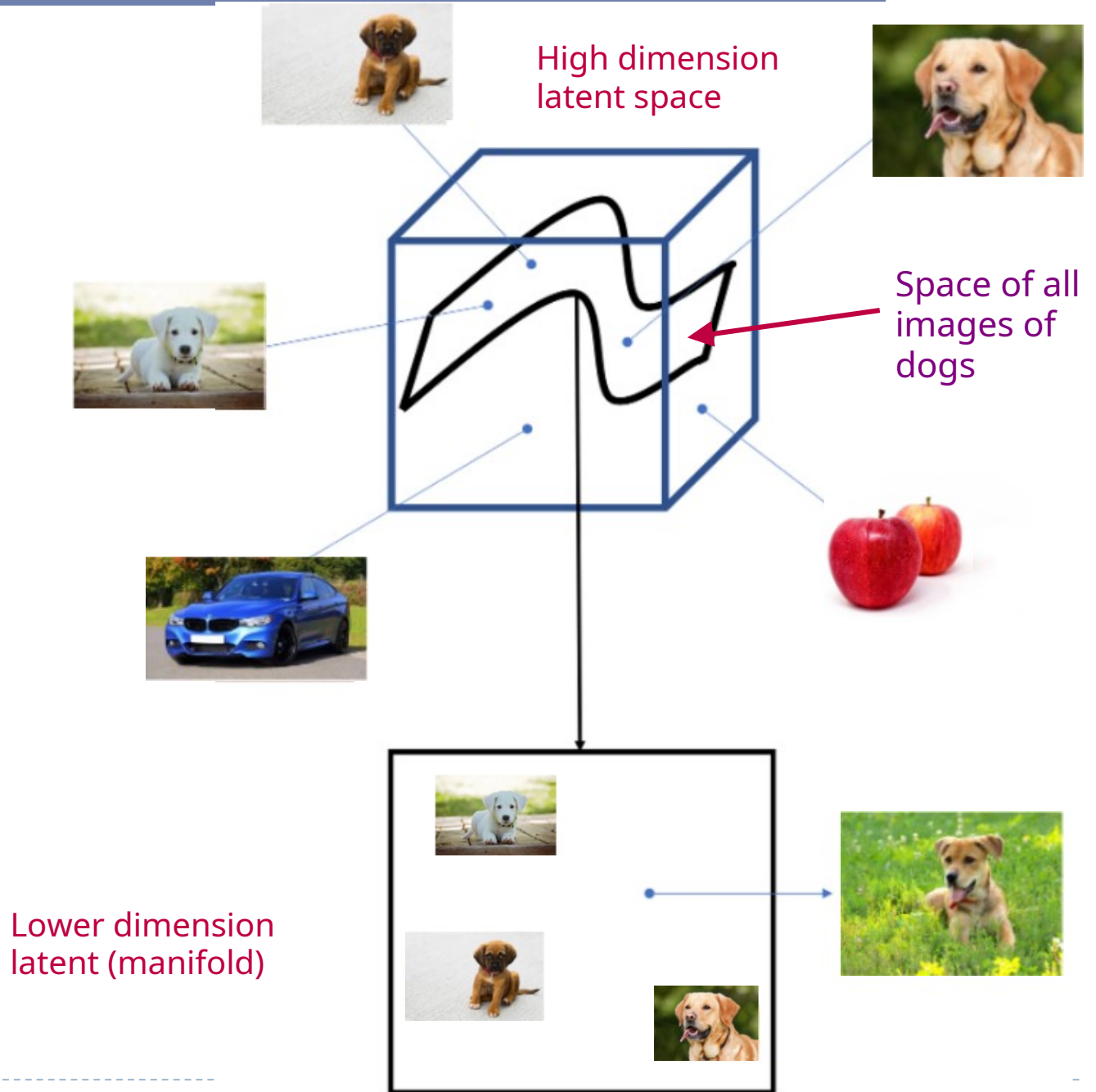
Index

- 5C.1 Introduction to GANs
- 5C.2 A simple example: 2D model
- 5C.3 More complex GANs
- 5C.4 GANs challenges and problems
- 5C.5 WGANs: Wasserstein GANs
- 5C.6 Other GANs architectures
- 5C.7 GANs evaluation

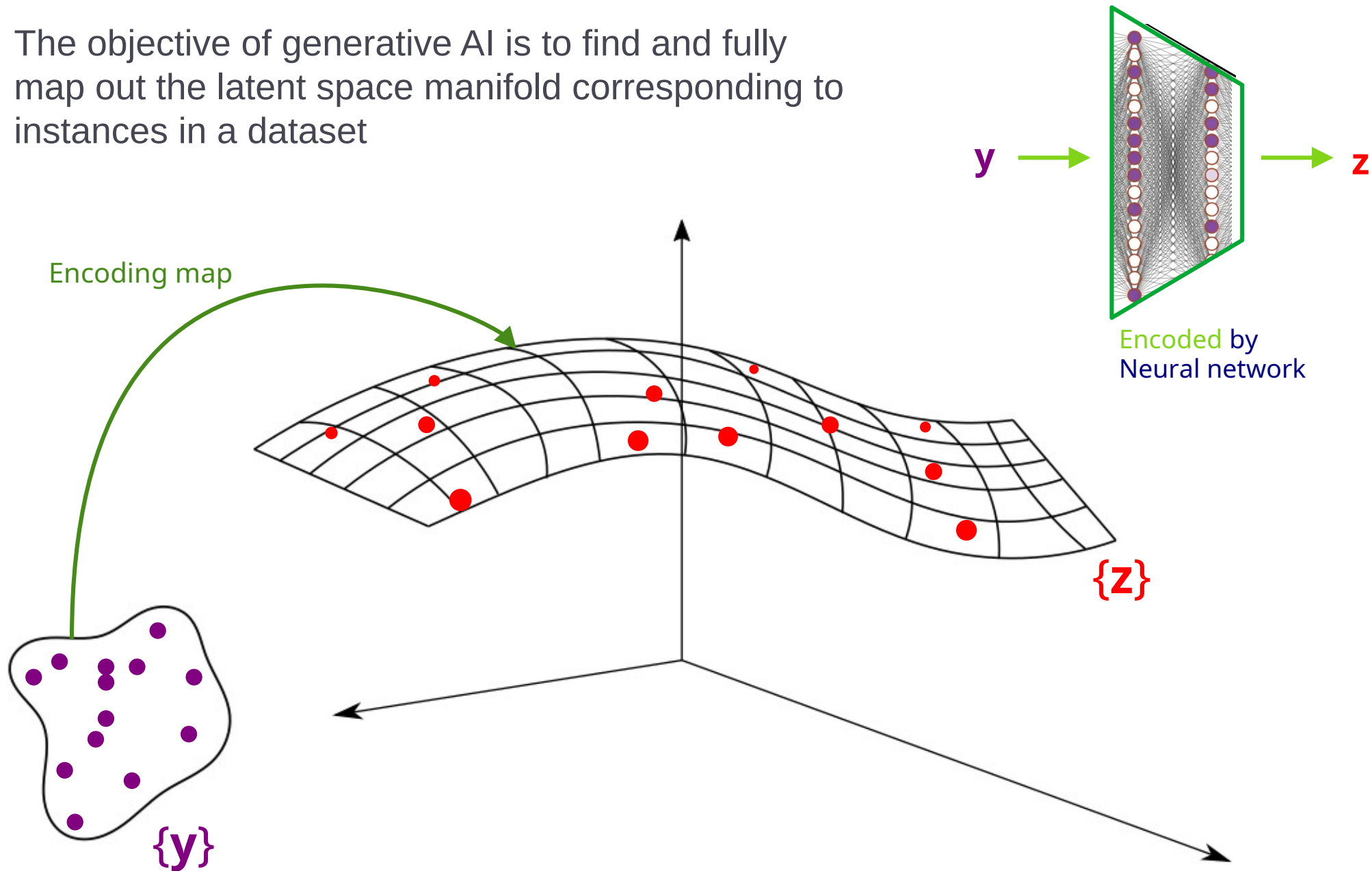
5C.1 Introduction to GANs

Latent Space:

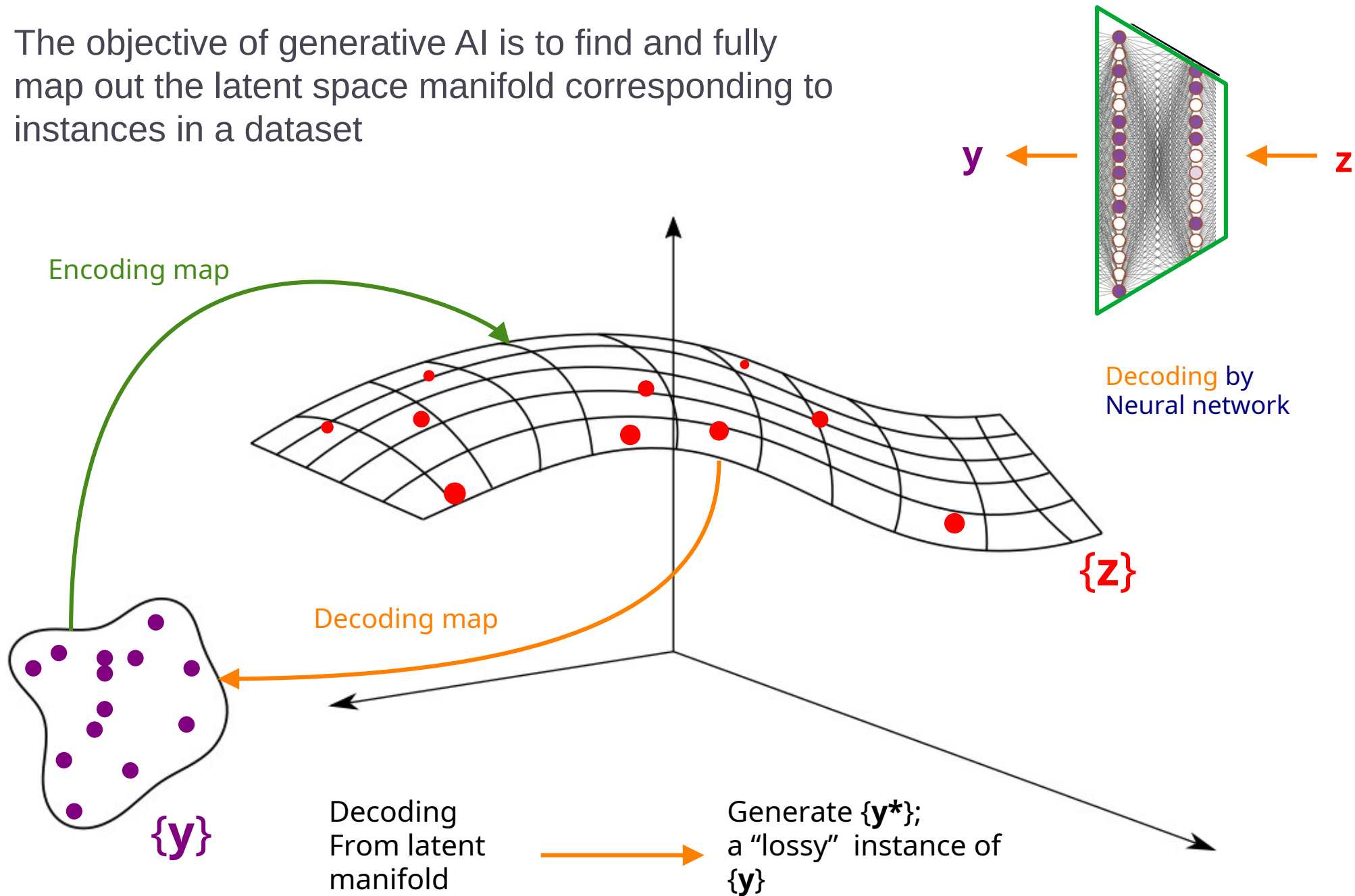
- A multi-dimensional vector space that represents hidden or unobserved features or attributes: *abstract space from which data can be sampled to produce new, similar data.*
- It may be learned during the training of a model and can be crucial for the generation or transformation of data.



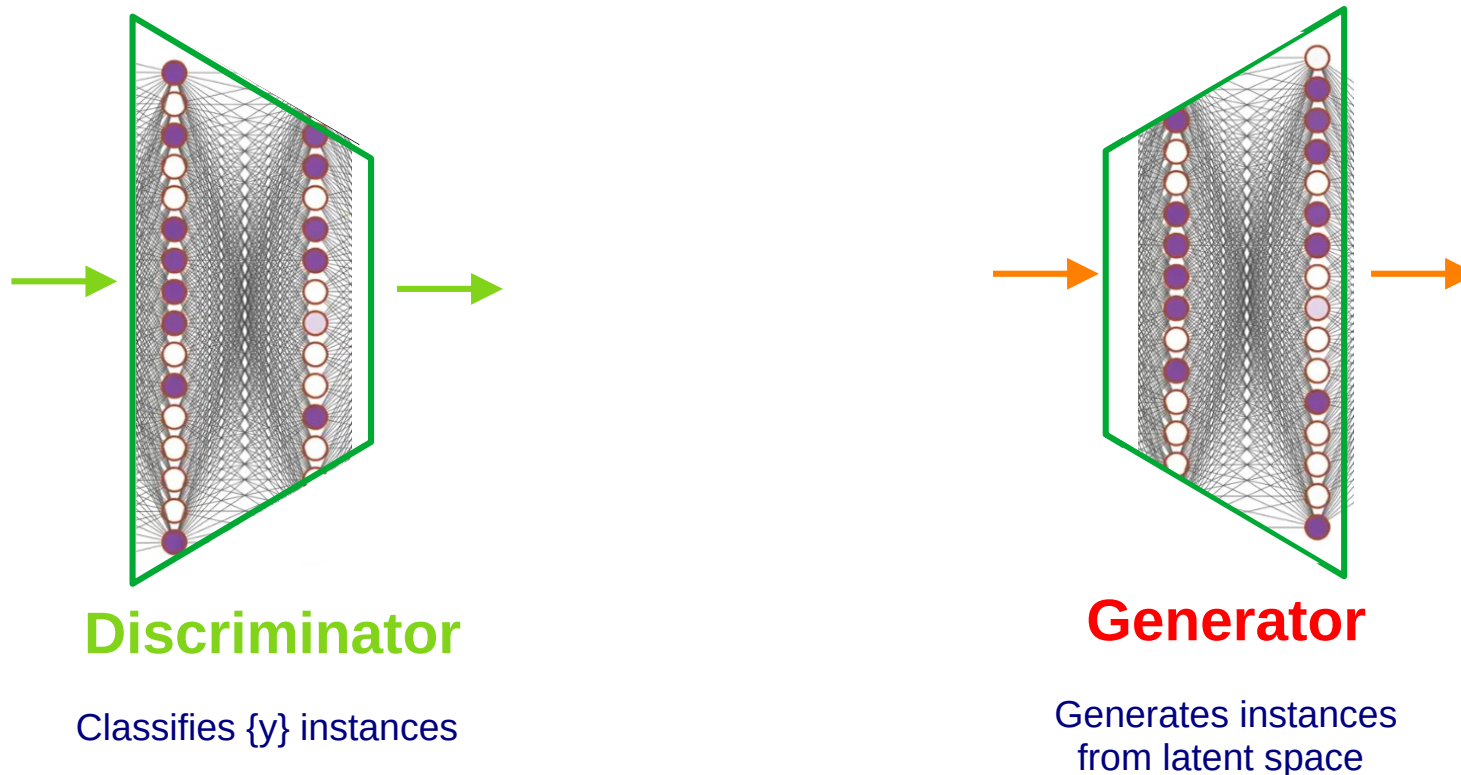
The objective of generative AI is to find and fully map out the latent space manifold corresponding to instances in a dataset



The objective of generative AI is to find and fully map out the latent space manifold corresponding to instances in a dataset



GANs (Generative Adversarial Networks): there are 2 “players”

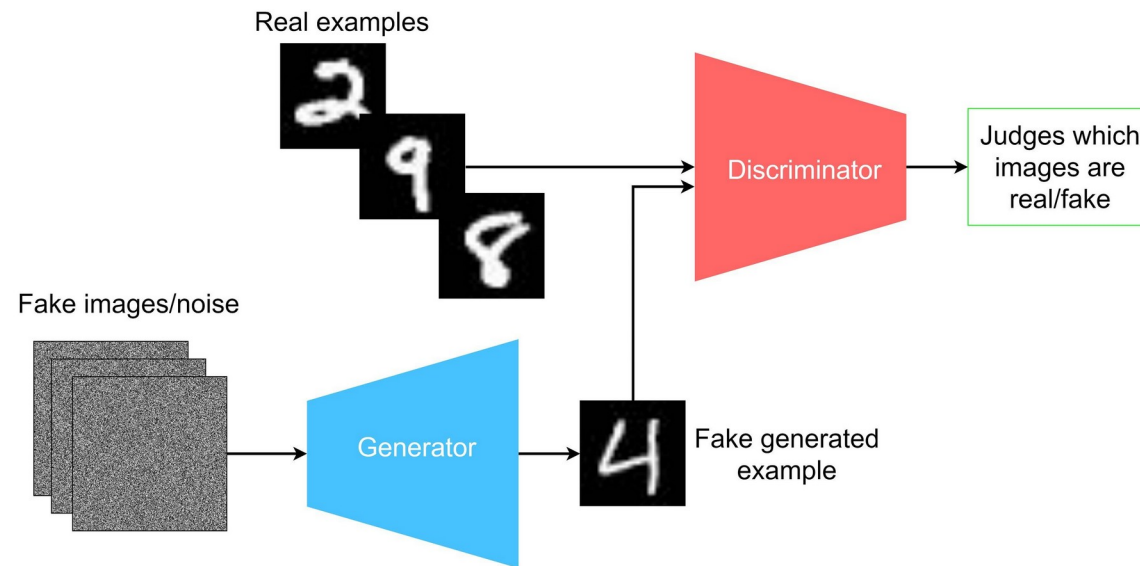


Training strategy (i.e., finding the latent space manifold):

use a Min-Max game: a zero-sum game between the two models

(models weights found together)

After Training: only keep the **Generator**



GAN training:

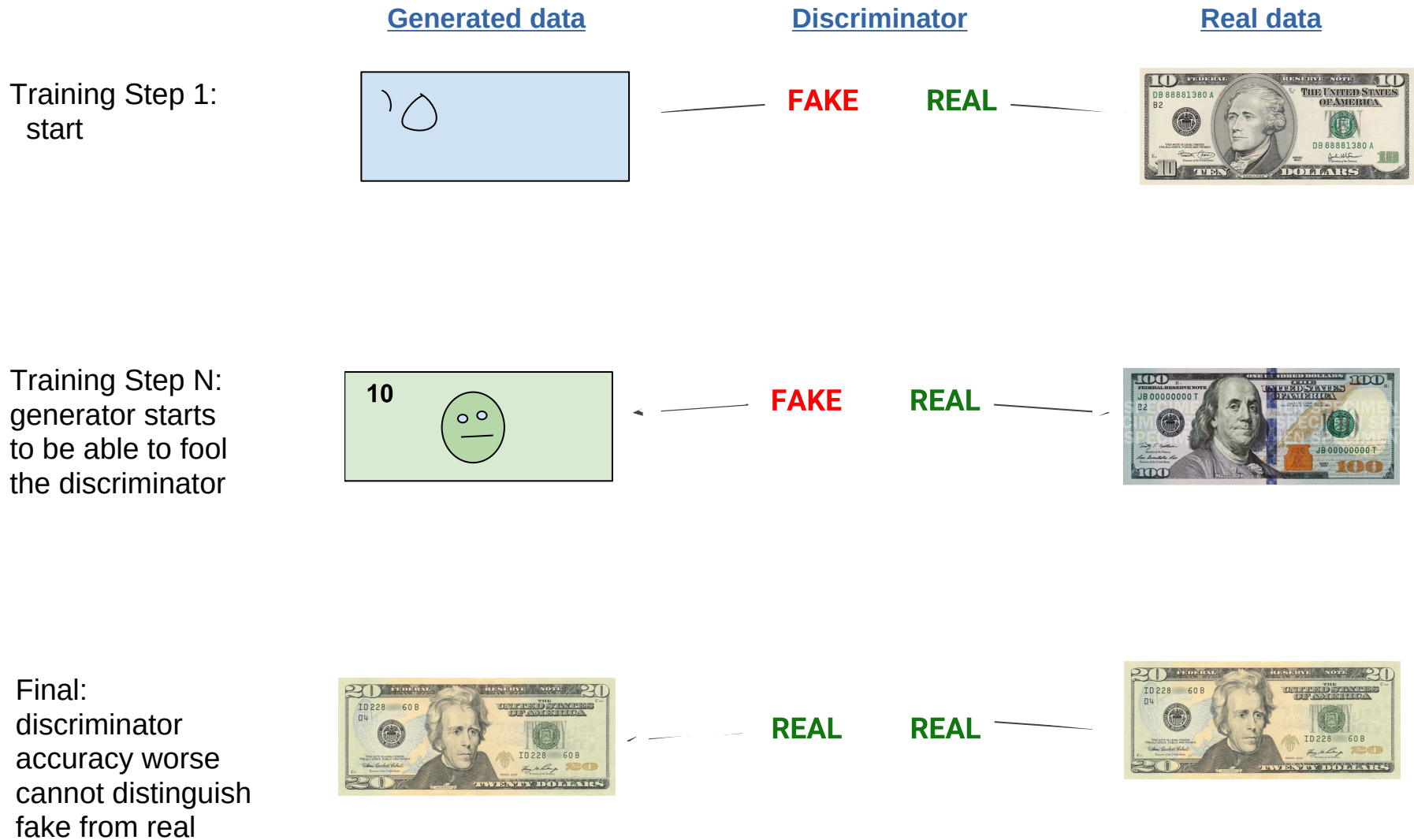
- Generator: learns to generate plausible data
- Discriminator: learns to distinguish the generator's fake data from the real data

The discriminator penalizes the generator for producing implausible results.

When training begins, the generator produces obviously fake data, and the discriminator quickly learns to tell that it's fake:



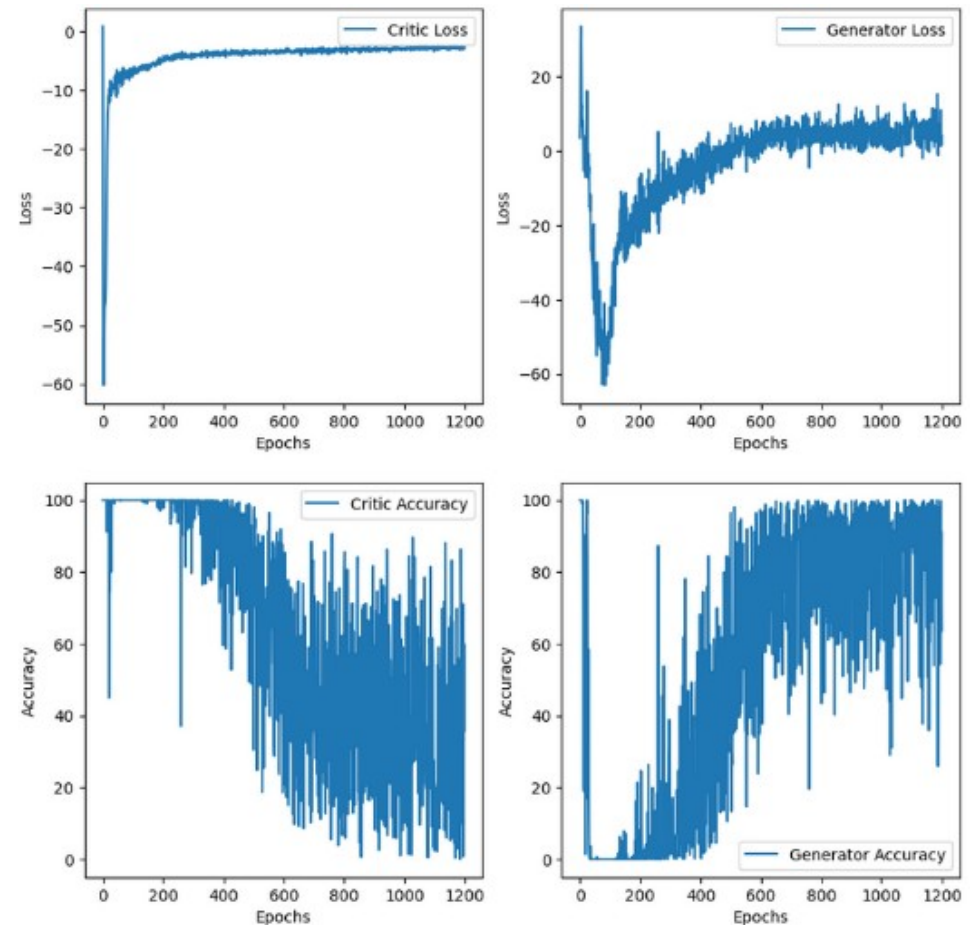
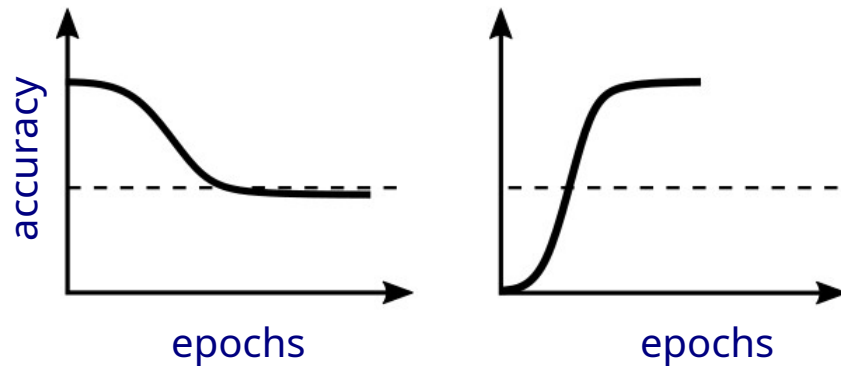
Training process



- **GAN: two separate networks:**
 - **Generator network** generates synthetic data samples, while
 - **Discriminator network** tries to distinguish the synthetic samples from real samples drawn from the training dataset.

Real Life Training

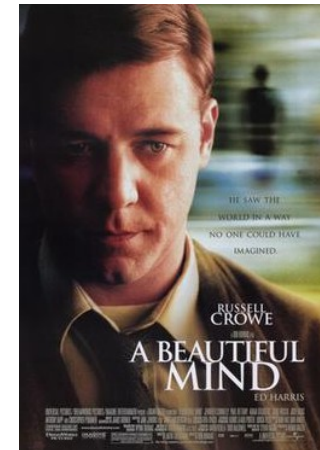
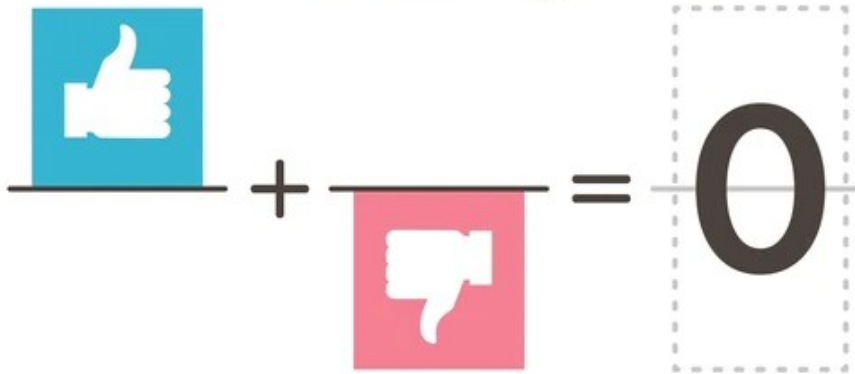
Ideal Training



A GAN is formally a zero-sum game

Zero-sum game is a [mathematical representation](#) in [game theory](#) and [economic theory](#) of a situation that involves two sides, where the result is an advantage for one side and an equivalent loss for the other.^[1] In other words, player one's gain is equivalent to player two's loss, with the result that the net improvement in benefit of the game is zero.^[2]

Zero-sum games are most often solved with the [minimax theorem](#) which is closely related to [linear programming duality](#),^[5] or with [Nash equilibrium](#). [Prisoner's Dilemma](#) is a classical non-zero-sum game.^[6]



A GAN is formally a zero-sum game with an error composed by two terms:

Discriminator (D) loss :

The discriminator's goal is to correctly classify real samples as real and fake samples (produced by the generator) as fake. Its loss is typically represented as:

$$L_D = -\frac{1}{2} \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log(D(x))] - \frac{1}{2} \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

1 → real sample
0 → fake sample

Generator (G) loss:

The generator's goal is to produce samples that the discriminator incorrectly classifies as real. Its loss is typically represented as:

$$L_G = -\frac{1}{2} \mathbb{E}_{z \sim p_z(z)} [\log(D(G(z)))]$$

The Generator
wants this value to
be close to 1

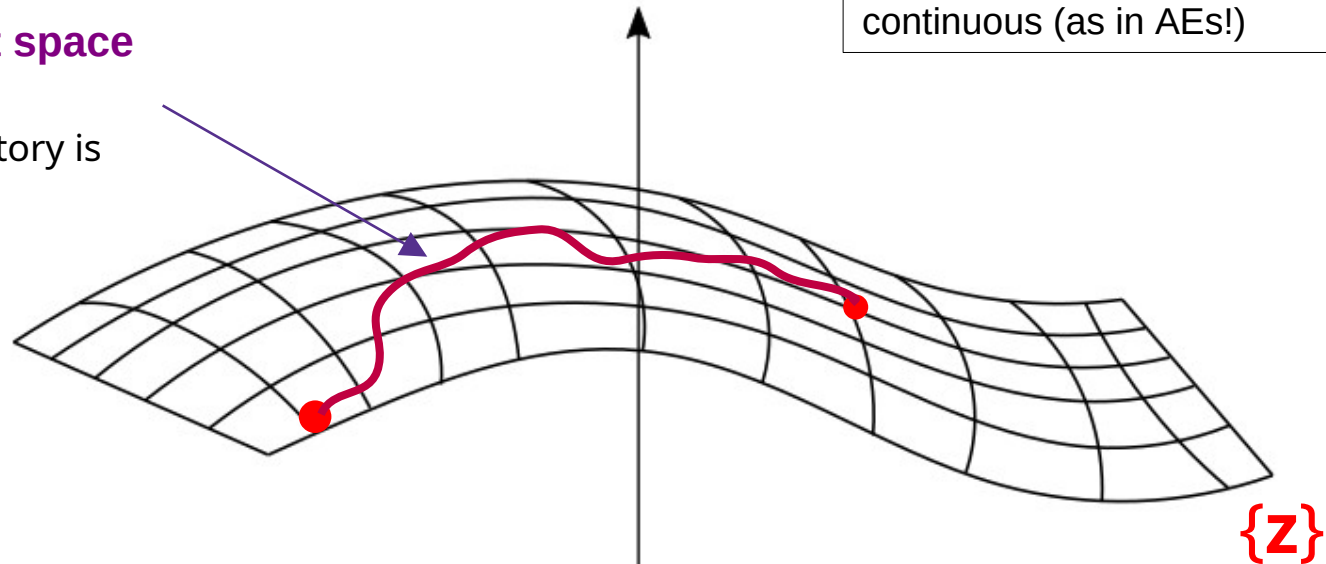
$$L_{\text{GAN}} = \min_G \max_D L_D + L_G$$



What is all this for?

Walking the latent space

Any point on the trajectory is in the distribution $\{z\}$



Key Idea:

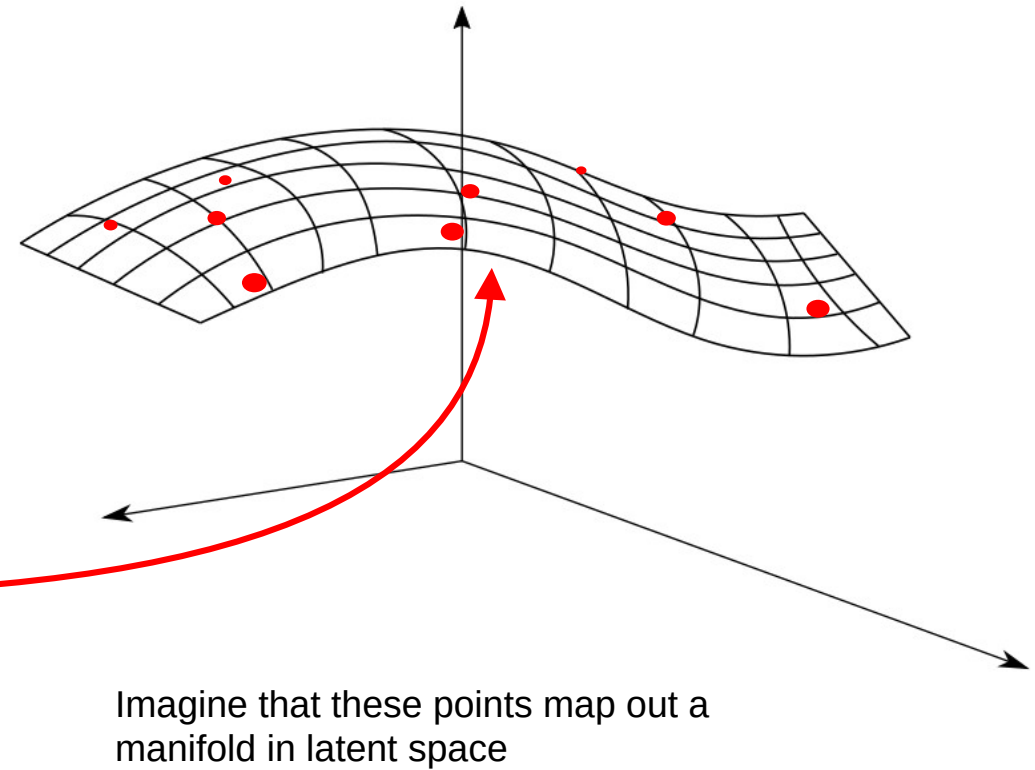
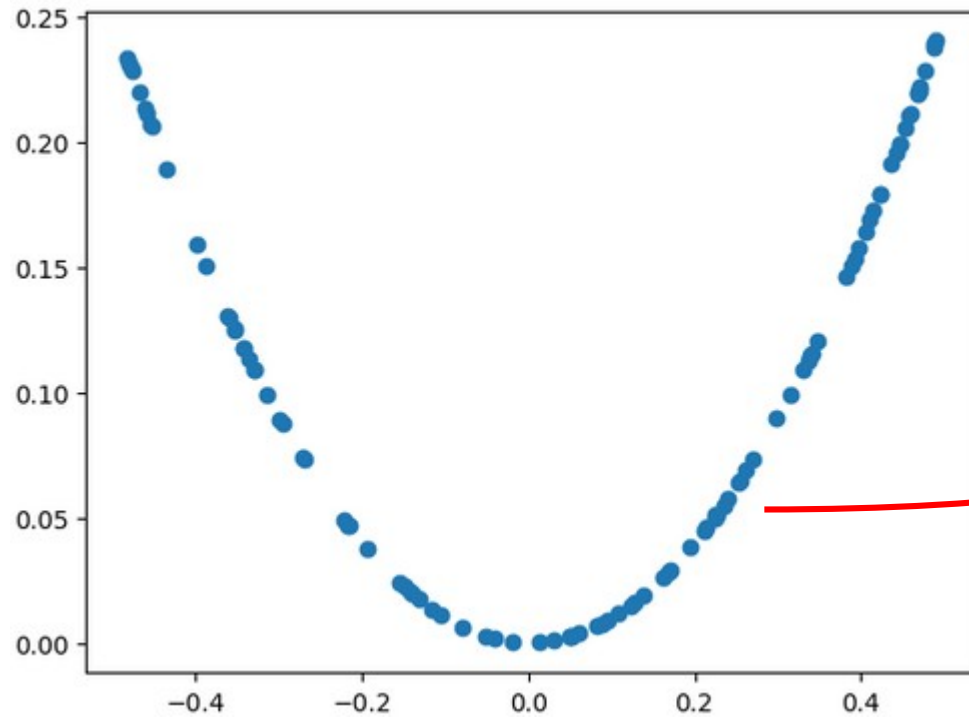
If we have $\{z\}$ we can sample from it and generate any point on its surface

A property of GANs:

- the latent values z that are learned tend to be well behaved when you manipulate them mathematically.
- take our two noise samples, we can interpolate between the noise vectors (say, 50% of one and 50% of the other) to produce interpolations of the images



5C.2 A simple example: 2D model

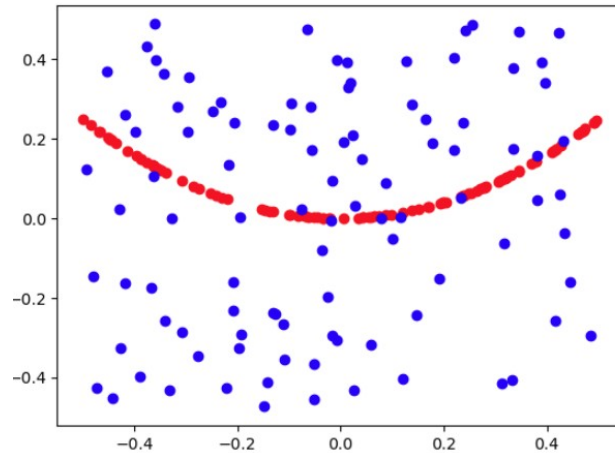


Objective to find the **Generator** so that we can generate any point in this distribution

Starting with a random distribution.

The **Generator** and the **Discriminator** are trained to discover the latent manifold:

- all the red points are “approved” by the Discriminator
- any points can be generated but the Generator’s goal is to generate good (red) points.



```
from numpy.random import rand
from numpy import hstack
from matplotlib import pyplot
import numpy as np

# generate randoms sample from x^2
def generate_samples(n=100):
    # generate random inputs in [-0.5, 0.5]
    X1 = rand(n) - 0.5
    # generate outputs X^2 (quadratic)
    X2 = X1 * X1
    # stack arrays
    X1 = X1.reshape(n, 1)
    X2 = X2.reshape(n, 1)
    return hstack((X1, X2))

# generate samples
data = generate_samples()
data2 = np.random.uniform(-0.5, 0.5, size=(100, 2))

# plot samples
pyplot.scatter(data[:, 0], data[:, 1], color='red')
pyplot.scatter(data2[:, 0], data2[:, 1], color='blue')

pyplot.show()
```

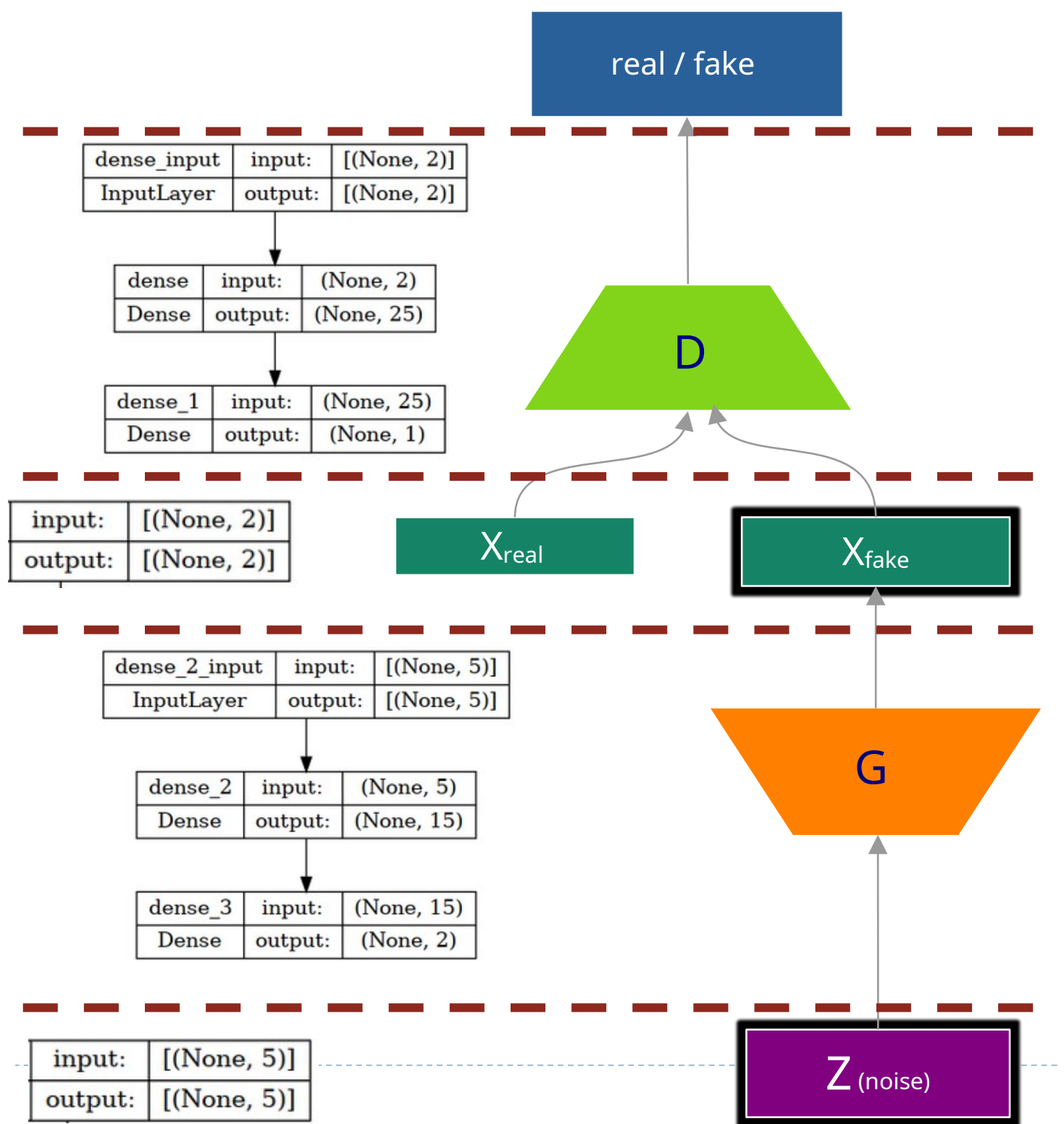
```
# generate points in latent space as input for the generator
def generate_latent_points(latent_dim, n):
    # generate points in the latent space
    x_input = randn(latent_dim * n)
    # reshape into a batch of inputs for the network
    x_input = x_input.reshape(n, latent_dim)
    return x_input

# use the generator to generate n fake examples, with class labels
def generate_fake_samples(generator, latent_dim, n):
    # generate points in latent space
    x_input = generate_latent_points(latent_dim, n)
    # predict outputs
    X = generator.predict(x_input)
    # create class labels
    y = zeros((n, 1))
    return X, y
```

In this example, we are going to use a 5-D latent space (latent_dim = 5)

The GAN

2d points:



A simple implementation

Inputs: Sample with two real values.

Outputs: Binary classification, likelihood the sample is real (or fake)

Inputs: Point in latent space, e.g. a five-element vector of Gaussian random numbers.

Outputs: Two-element vector representing a generated sample for our function (hopefully x and x^2).

```
# define the standalone discriminator model
def define_discriminator(n_inputs=2):
    model = Sequential()
    model.add(Dense(25, activation='relu', kernel_initializer='he_uniform', input_dim=n_inputs))
    model.add(Dense(1, activation='sigmoid'))
    # compile model
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model

# define the standalone generator model
def define_generator(latent_dim, n_outputs=2):
    model = Sequential()
    model.add(Dense(15, activation='relu', kernel_initializer='he_uniform', input_dim=latent_dim))
    model.add(Dense(n_outputs, activation='linear'))
    return model
```

Discriminator

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 25)	75
dense_5 (Dense)	(None, 1)	26
Total params: 101		
Trainable params: 101		
Non-trainable params: 0		

Generator

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 15)	90
dense_1 (Dense)	(None, 2)	32
Total params: 122		
Trainable params: 122		
Non-trainable params: 0		

Putting the two networks together: GAN

```
# define the combined generator and discriminator model, for updating the generator
def define_gan(generator, discriminator):
    # make weights in the discriminator not trainable
    discriminator.trainable = False
    # connect them
    model = Sequential()
    # add generator
    model.add(generator)
    # add the discriminator
    model.add(discriminator)
    # compile model
    model.compile(loss='binary_crossentropy', optimizer='adam')
    return model
```

dense_input	input:	[(None, 2)]
InputLayer	output:	[(None, 2)]

dense	input:	(None, 2)
Dense	output:	(None, 25)

dense_1	input:	(None, 25)
Dense	output:	(None, 1)



dense_2_input	input:	[(None, 5)]
InputLayer	output:	[(None, 5)]

dense_2	input:	(None, 5)
Dense	output:	(None, 15)

dense_3	input:	(None, 15)
Dense	output:	(None, 2)



Training: playing the min-max game

```
# train the generator and discriminator
def train(g_model, d_model, gan_model, latent_dim, n_epochs=10000, n_batch=128, n_eval=400):
    # determine half the size of one batch, for updating the discriminator
    half_batch = int(n_batch / 2)
    # manually enumerate epochs
    for i in range(n_epochs):
        # prepare real samples
        x_real, y_real = generate_real_samples(half_batch)
        # prepare fake examples
        x_fake, y_fake = generate_fake_samples(g_model, latent_dim, half_batch)
        # update discriminator
        d_model.train_on_batch(x_real, y_real)
        d_model.train_on_batch(x_fake, y_fake)
        # prepare points in latent space as input for the generator
        x_gan = generate_latent_points(latent_dim, n_batch)
        # create inverted labels for the fake samples
        y_gan = ones((n_batch, 1))
        # update the generator via the discriminator's error
        gan_model.train_on_batch(x_gan, y_gan)
        # evaluate the model every n_eval epochs
        if (i+1) % n_eval == 0:
            summarize_performance(i, g_model, d_model, latent_dim)
```



1. Batch for discriminator: half real and half fake
2. Update the discriminator
3. Batch for generator: fake
4. Update whole model (D is “frozen”)

Done by “**Freezing the weights**”

This step is where we are minimizing this loss function:

$$L_D = -\frac{1}{2} \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log(D(x))] - \frac{1}{2} \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

$$L_G = -\frac{1}{2} \mathbb{E}_{z \sim p_z(z)} [\log(D(G(z)))]$$

$$L_{\text{GAN}} = \min_G \max_D L_D + L_G$$

GAN freezing

Discriminator:

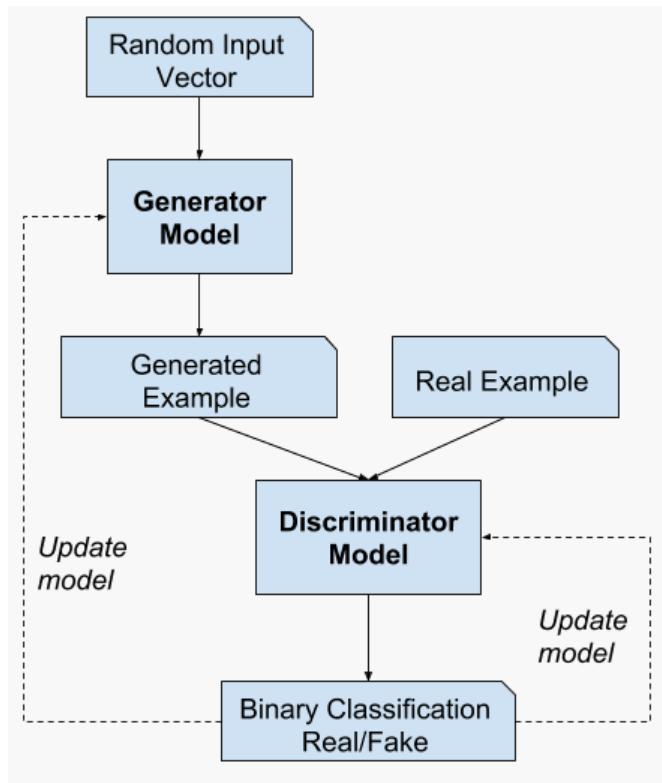
- Trains on real data (targets as 1s) and fake data generated by the generator (targets as 0s)
- Goal: Accurately distinguish between real and generated samples.

Generator:

- Not compiled on its own; **it's trained via the combined GAN model.**
- Produces output aimed to fool the discriminator.

```
# define the standalone discriminator model
def define_discriminator(n_inputs=2):
    model = Sequential()
    model.add(Dense(25, activation='relu', kernel_initializer='he_uniform', input_dim=n_inputs))
    model.add(Dense(1, activation='sigmoid'))
    # compile model
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model

# define the standalone generator model
def define_generator(latent_dim, n_outputs=2):
    model = Sequential()
    model.add(Dense(15, activation='relu', kernel_initializer='he_uniform', input_dim=latent_dim))
    model.add(Dense(n_outputs, activation='linear'))
    return model
```



Combined GAN Model:

- Composed of both the generator and discriminator linked sequentially.
- During the training of this model, the discriminator's weights are frozen (discriminator.trainable = False).

```
# define the combined generator and discriminator model, for updating the generator
def define_gan(generator, discriminator):
    # make weights in the discriminator not trainable
    discriminator.trainable = False
    # connect them
    model = Sequential()
    # add generator
    model.add(generator)
    # add the discriminator
    model.add(discriminator)
    # compile model
    model.compile(loss='binary_crossentropy', optimizer='adam')
    return model
```


Discriminator: only concerned with distinguishing between real and fake examples; therefore, can be trained in a standalone manner on examples of each.

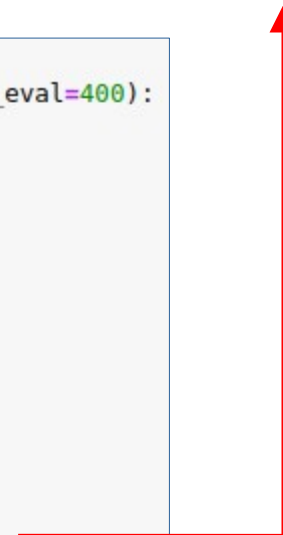
Generator model: only concerned with the discriminator's performance on fake examples. Therefore, we will mark all of the layers in the discriminator as not trainable when it is part of the GAN model so that they can not be updated and overtrained on fake examples.

Zero-sum game “training”

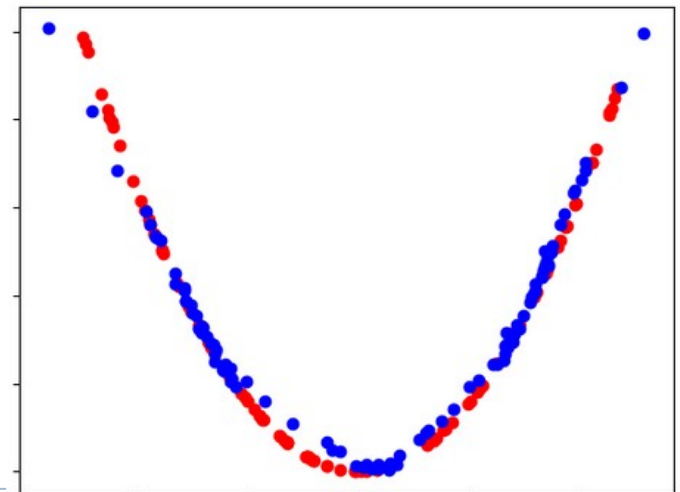
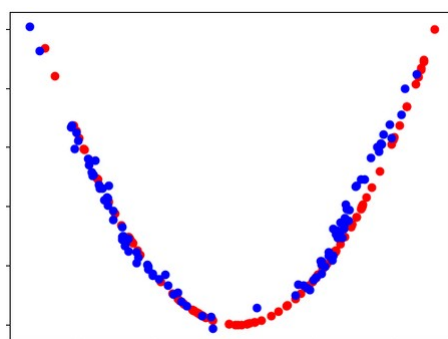
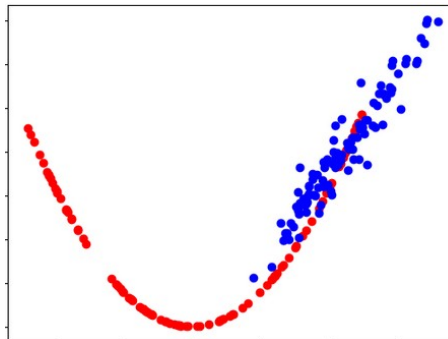
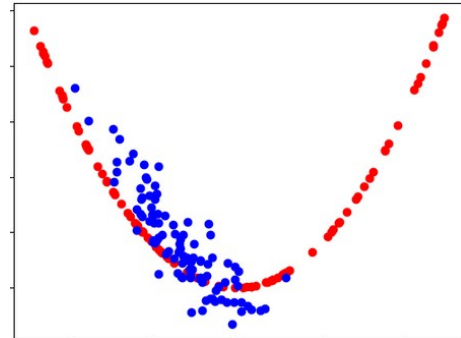
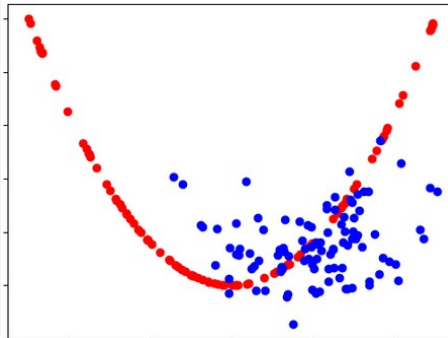
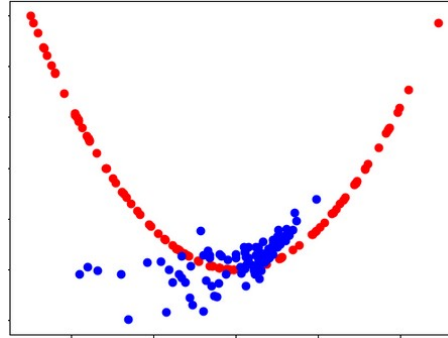
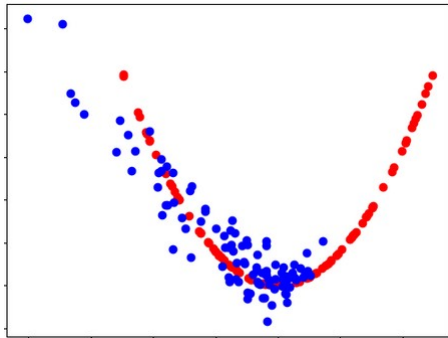
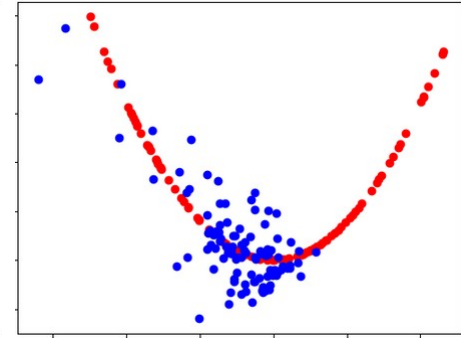
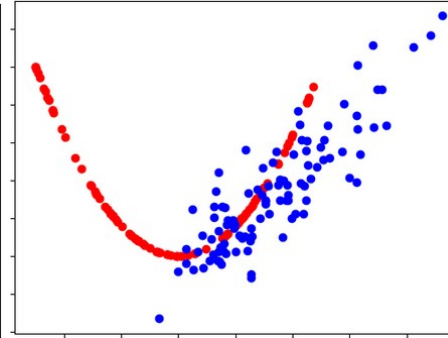
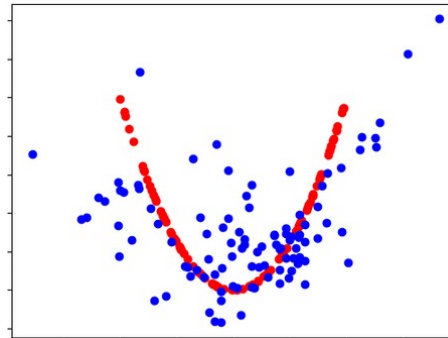
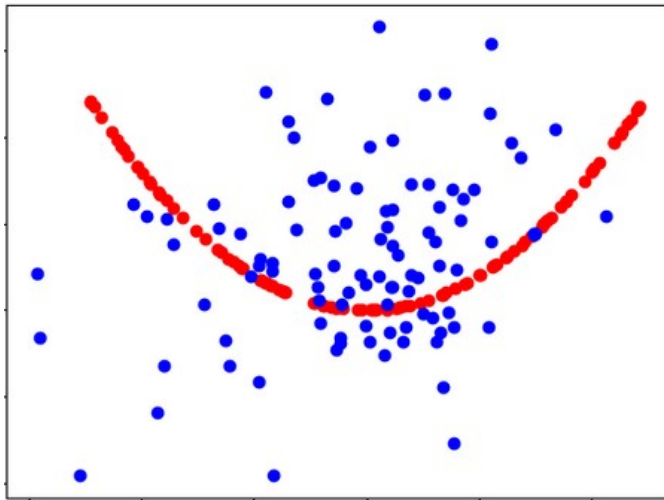
Training generator within subsumed GAN model

- Make Real: make discriminator believe generated samples are real; mark generated samples as real (class 1) during generator training
- Backpropagation identifies large errors
 - Updates only generator's model weights to correct errors
 - Results in improved, more plausible fake samples

```
# train the generator and discriminator
def train(g_model, d_model, gan_model, latent_dim, n_epochs=10000, n_batch=128, n_eval=400):
    # determine half the size of one batch, for updating the discriminator
    half_batch = int(n_batch / 2)
    # manually enumerate epochs
    for i in range(n_epochs):
        # prepare real samples
        x_real, y_real = generate_real_samples(half_batch)
        # prepare fake examples
        x_fake, y_fake = generate_fake_samples(g_model, latent_dim, half_batch)
        # update discriminator
        d_model.train_on_batch(x_real, y_real)
        d_model.train_on_batch(x_fake, y_fake)
        # prepare points in latent space as input for the generator
        x_gan = generate_latent_points(latent_dim, n_batch)
        # create inverted labels for the fake samples
        y_gan = ones((n_batch, 1))
        # update the generator via the discriminator's error
        gan_model.train_on_batch(x_gan, y_gan)
        # evaluate the model every n_eval epochs
        if (i+1) % n_eval == 0:
            summarize_performance(i, g_model, d_model, latent_dim)
```



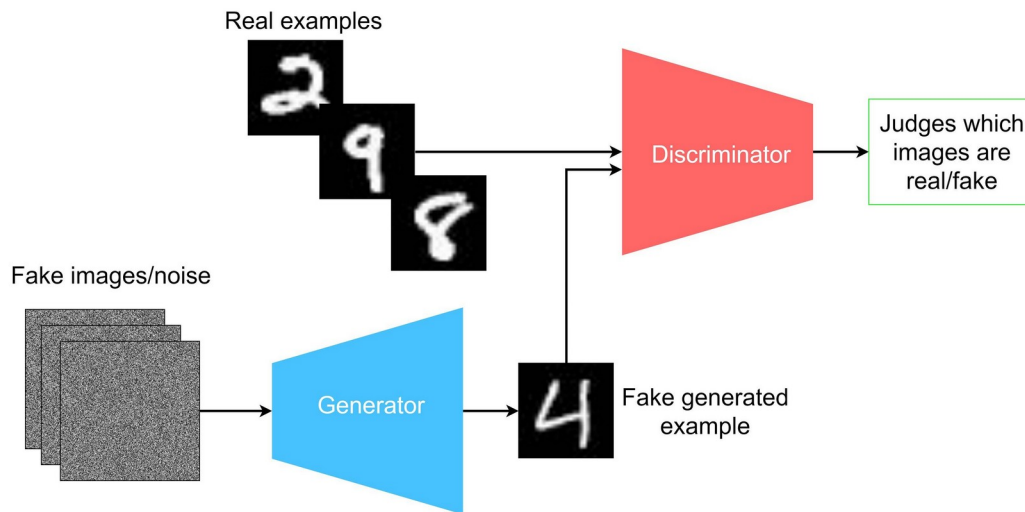
Results: after lots of epochs



The real (blue) and the “fake” (red) samples are undistinguishable

5C.3 More complex GANs

What about the “Hello world” of AI?



Could we generate hand-written digits?


```
def create_generator():
    model = Sequential()
    model.add(Dense(256, input_dim=random_dim))
    model.add(LeakyReLU(0.2))
    model.add(Dense(512))
    model.add(LeakyReLU(0.2))
    model.add(Dense(1024))
    model.add(LeakyReLU(0.2))
    model.add(Dense(784, activation='sigmoid'))
    model.compile(loss='binary_crossentropy',
                  optimizer=Adam(learning_rate=0.0002, beta_1=0.5))
    return model
```

```
def create_discriminator():
    model = Sequential()
    model.add(Dense(1024, input_dim=784))
    model.add(LeakyReLU(0.2))
    model.add(Dropout(0.3))
    model.add(Dense(512))
    model.add(LeakyReLU(0.2))
    model.add(Dropout(0.3))
    model.add(Dense(256))
    model.add(LeakyReLU(0.2))
    model.add(Dropout(0.3))
    model.add(Dense(1, activation='sigmoid'))
    model.compile(loss='binary_crossentropy',
                  optimizer=Adam(learning_rate=0.0002, beta_1=0.5))
    return model
```

random_dim = 100
epochs = 200
batch_size = 128

```
# For combined model training
discriminator.trainable = False
gan_input = Input(shape=(random_dim,))
x = generator(gan_input)
gan_output = discriminator(x)
gan = Model(gan_input, gan_output)
gan.compile(loss='binary_crossentropy', optimizer=Adam(learning_rate=0.0002, beta_1=0.5))
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 256)	25,856
leaky_re_lu (LeakyReLU)	(None, 256)	0
dense_1 (Dense)	(None, 512)	131,584
leaky_re_lu_1 (LeakyReLU)	(None, 512)	0
dense_2 (Dense)	(None, 1024)	525,312
leaky_re_lu_2 (LeakyReLU)	(None, 1024)	0
dense_3 (Dense)	(None, 784)	803,600

Total params: 1,486,352 (5.67 MB)

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 1024)	803,840
leaky_re_lu_3 (LeakyReLU)	(None, 1024)	0
dropout (Dropout)	(None, 1024)	0
dense_5 (Dense)	(None, 512)	524,800
leaky_re_lu_4 (LeakyReLU)	(None, 512)	0
dropout_1 (Dropout)	(None, 512)	0
dense_6 (Dense)	(None, 256)	131,328
leaky_re_lu_5 (LeakyReLU)	(None, 256)	0
dropout_2 (Dropout)	(None, 256)	0
dense_7 (Dense)	(None, 1)	257

Total params: 1,460,225 (5.57 MB)

```

d_losses = []
g_losses = []

batchCount = int(X_train.shape[0] / batch_size)

tf.keras.utils.disable_interactive_logging()
for epoch in range(epochs):
    for _ in range(batchCount):

        # Train Discriminator
        noise = np.random.normal(0, 1, size=[batch_size, random_dim])
        generated_images = generator.predict(noise)
        real_images = X_train[np.random.randint(0, X_train.shape[0], batch_size)]

        X = np.concatenate([real_images, generated_images])
        y_dis = np.zeros(2 * batch_size)
        y_dis[:batch_size] = 1

        # Reinitialize the optimizer and enable training for the discriminator
        discriminator.trainable = True
        d_loss = discriminator.train_on_batch(X, y_dis)

        # Train Generator
        noise = np.random.normal(0, 1, size=[batch_size, random_dim])
        y_gen = np.ones(batch_size)

        # Reinitialize the optimizer, disable training for the discriminator, and compile GAN
        discriminator.trainable = False
        g_loss = gan.train_on_batch(noise, y_gen)

    # Keep losses of last batch of each epoch
    d_losses.append(d_loss)
    g_losses.append(g_loss)

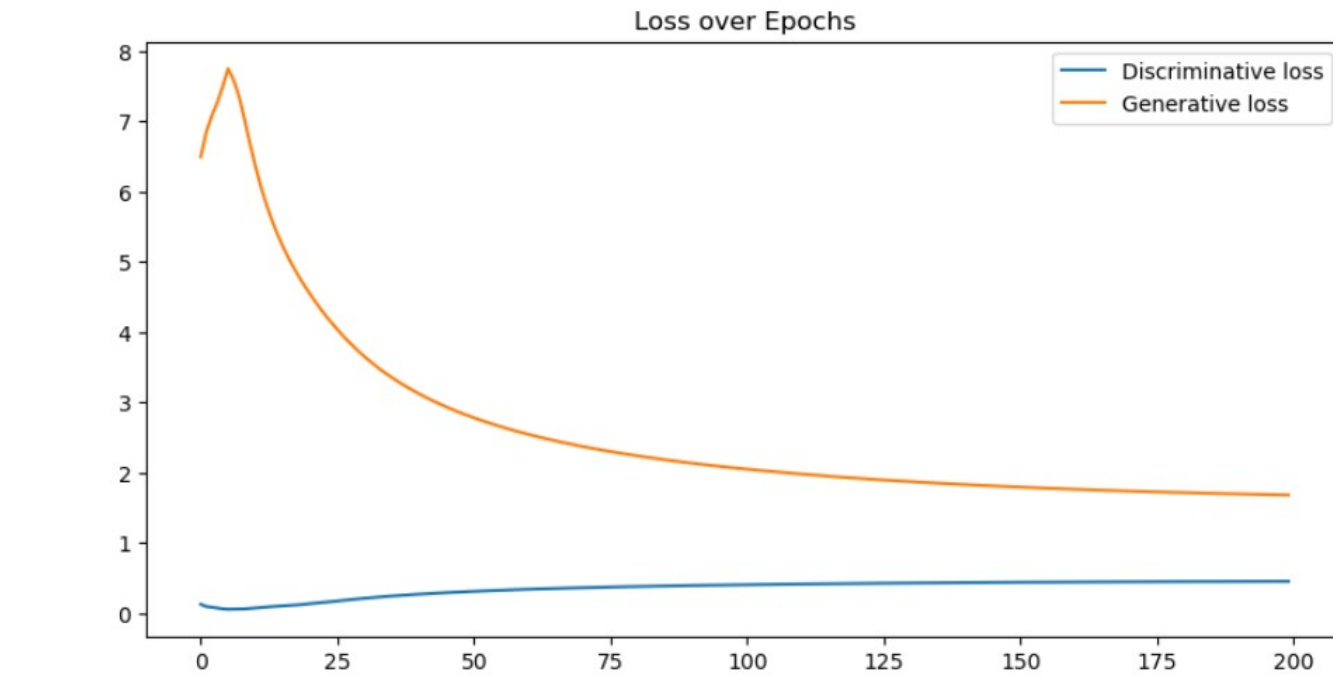
    # Progress logs
    if epoch % sample_interval == 0 or epochs==epoch:
        print(f"Epoch {epoch} / {epochs}, Discriminator Loss: {d_loss}, Generator Loss: {g_loss}")
        show_images(epochs, generator)

```

Batch: 128 real digits
+128 fake digits

Train only the generator
part of the GAN

Results



Epoch 0 / 200, Discriminator Loss: 0.12664425373077393, Generator Loss: 6.490775108337402



Epoch 20 / 200, Discriminator Loss: 0.13416872918605804, Generator Loss: 4.537552833557129



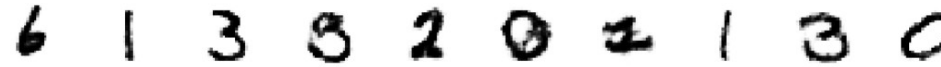
Epoch 40 / 200, Discriminator Loss: 0.270513117313385, Generator Loss: 3.117764711380005



Epoch 60 / 200, Discriminator Loss: 0.3405557870864868, Generator Loss: 2.541886806488037



Epoch 80 / 200, Discriminator Loss: 0.3800140619277954, Generator Loss: 2.23626756680908



Epoch 100 / 200, Discriminator Loss: 0.404946893453598, Generator Loss: 2.0476534366607666



Epoch 120 / 200, Discriminator Loss: 0.42221522331237793, Generator Loss: 1.9193971157073975



Epoch 140 / 200, Discriminator Loss: 0.43453049659729004, Generator Loss: 1.8279635906219482



Epoch 160 / 200, Discriminator Loss: 0.4432348608970642, Generator Loss: 1.7617621421813965



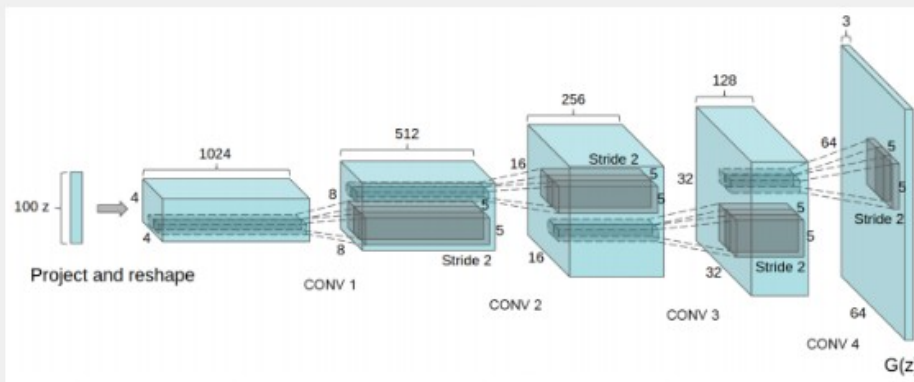
Epoch 180 / 200, Discriminator Loss: 0.44907158613204956, Generator Loss: 1.7140675783157349



Now more complex: creating faces with DCGANs (Deep Convolutional GANs)

DCGAN in Tensorflow

Tensorflow implementation of [Deep Convolutional Generative Adversarial Networks](#) which is a stabilize Generative Adversarial Networks. The referenced torch code can be found [here](#).



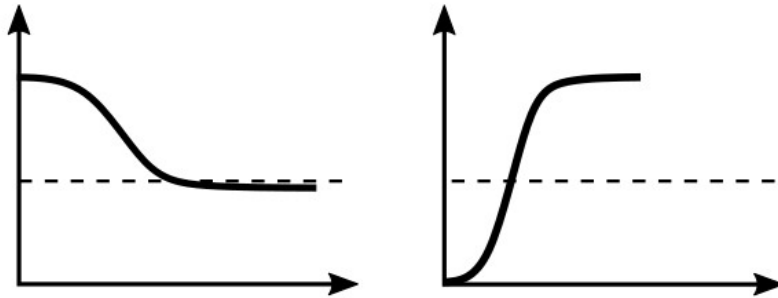
- [Brandon Amos](#) wrote an excellent [blog post](#) and [image completion code](#) based on this repo.
- To avoid the fast convergence of D (discriminator) network, G (generator) network is updated twice for each D network update, which differs from original paper.



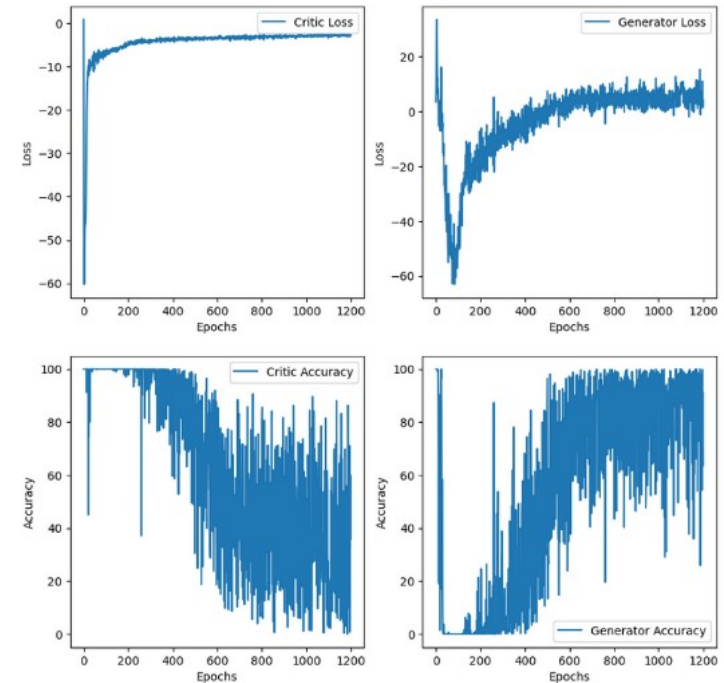
5C.4 GANs problems and challenges

Problem no. 1: failure to convergence

Ideal training



Real life training



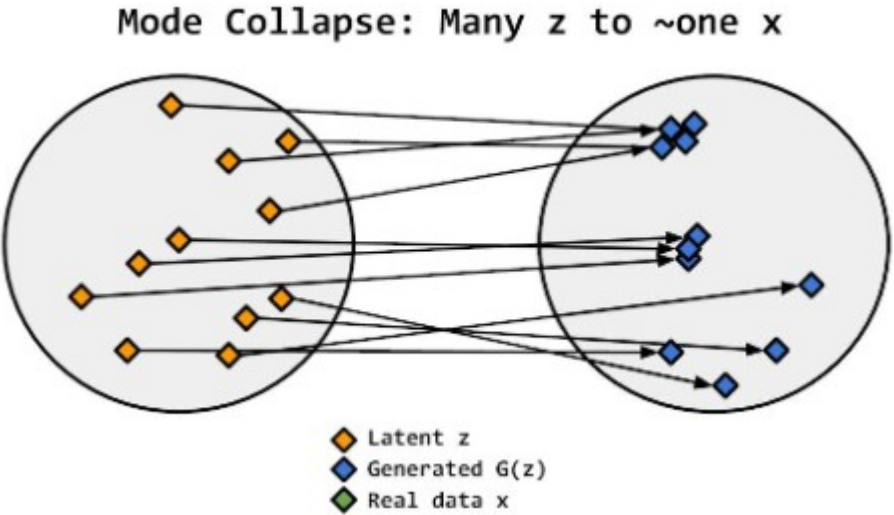
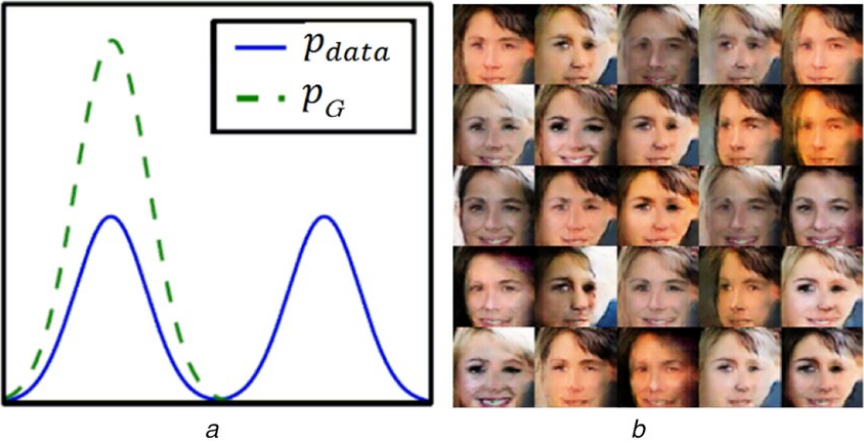
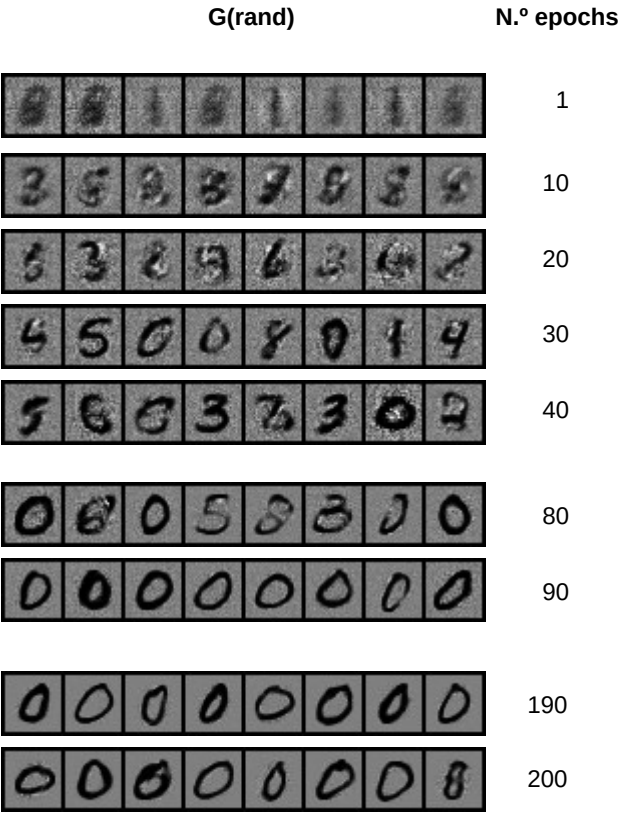
The job of the generator is a harder one!!

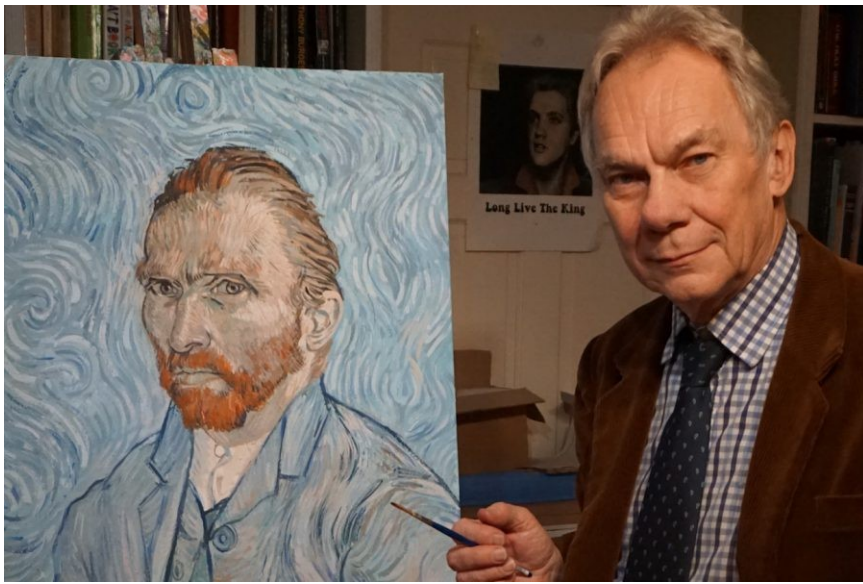
The discriminator dominates, but the feedback to the generator is not useful... quality of generated samples collapses.

The generator “oscillates” around an optimum

GANs are very sensible to hyperparameters!!!

Problem no. 2:
mode collapse





VS.

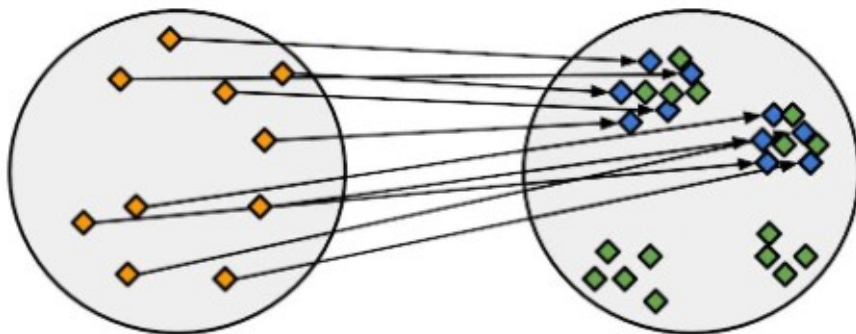


When **mode collapse** occurs, the generator loss can increase over time, and here's why:

- 1) **Decreasing Discriminator Error:** during mode collapse, the discriminator can easily tell that the generator is producing limited or repetitive outputs, so it becomes better at identifying fake samples. This makes the discriminator highly effective at rejecting the generator's outputs, increasing the generator's loss.
- 2) **Feedback Loop:** as the generator outputs the same (or similar) samples, the discriminator's feedback indicates that these samples are fake, and the generator responds by repeatedly trying to make similar samples slightly better, rather than generating new, diverse samples. This feedback loop reinforces the production of similar samples rather than diverse ones, which fails to reduce the generator's loss.
- 3) **Lack of Diversity Hurts Generator Performance:** because the generator is only learning a limited subset of outputs, it doesn't explore a wide enough range of potential data distributions, leading to a poorer approximation of the true data distribution. This increases the generator's loss since it's not generating realistic and varied samples as it's supposed to.
- 4) **Imbalanced Learning Process:** if the discriminator becomes too strong relative to the generator, it can lead to the generator "giving up" on certain variations and only producing a narrow subset of samples that it knows may have some chance of success. This imbalance prevents the generator from learning effectively, further increasing the generator's loss over time.

Problem no. 3: mode dropping

Mode Dropping: Real Data not in $G(z)$



- Mode collapse and Mode Dropping can co-occur
- Complete training collapse is a separate phenomenon, for which extreme Mode Dropping and/or Mode Collapse are often symptoms

Mode dropping: some modes of the data are not represented in the output of the model



Mode Dropping on MNIST (Source:TripletGAN)

There are no 8's !!!

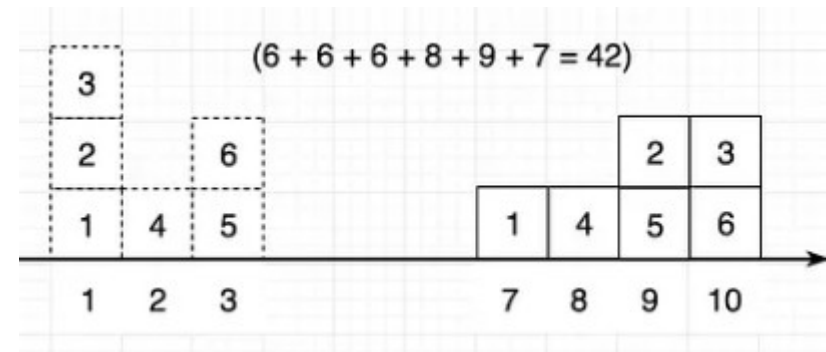
Training GANs is very challenging!

One of the main behind these problems is vanishing gradients!!

5C.5 WGANs: Wasserstein GANs

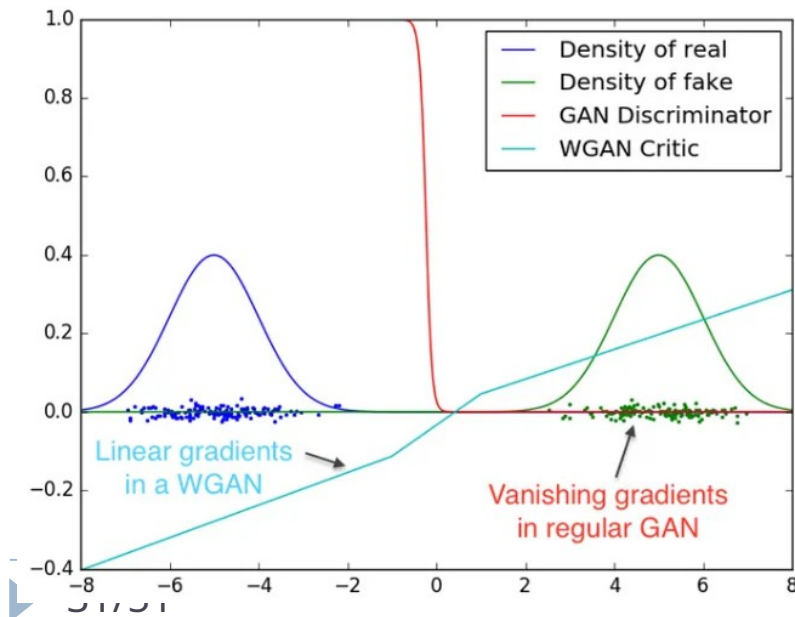
Wasserstein loss: measure of distance between two probability distributions

- Also known as **Earth Mover's Distance (EMD)**
- Used as loss functions in GAN training to stabilize and improve samples quality
- It is not a real vs fake decision, but a smooth distance
- It works better even where distributions do not overlap

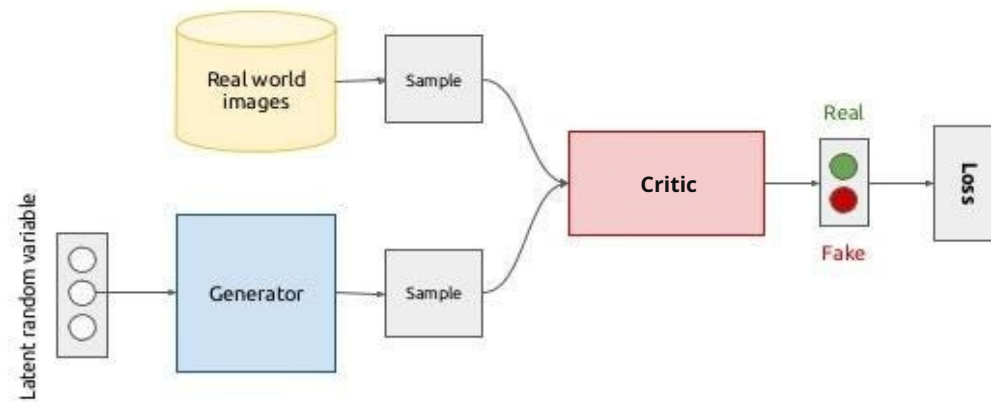


$$L_{\text{WGAN}} = \mathbb{E}_{x \sim \text{real}}[f(x)] - \mathbb{E}_{z \sim p_z}[f(G(z))]$$

Wasserstein GANs are less vulnerable to getting stuck than minimax-based GANs, and avoid problems with vanishing gradients. The earth mover distance also has the advantage of being a true metric: a measure of distance in a space of probability distributions. Cross-entropy is not a metric in this sense.



Change: now we don't have a discriminator, but a **critic**



The critic measures the distance of the distributions between the real and the fake samples

Wasserstein distance can have any value $(-\infty \dots \infty)$ (no activation function)

Important to ensure a constraint on how quickly the function's values can change, ensuring that the function does not have abrupt variation. We need a 1-Lipschitz function:

$$|f(x) - f(y)| \leq |x - y|$$

The change in the function's value between any two points does not exceed the distance between these two points

One possibility to achieve this is using **weight clipping** in the discriminator

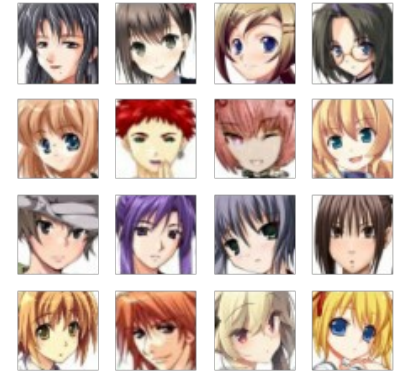
Example: generate Anime faces using a WGAN

```
# latent dimension of the random noise  
LATENT_DIM = 128
```

Model: "generator"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32768)	4227072
re_lu (ReLU)	(None, 32768)	0
reshape (Reshape)	(None, 8, 8, 512)	0
conv2d_transpose (Conv2DTranspose)	(None, 16, 16, 256)	2097152
batch_normalization (Batch Normalization)	(None, 16, 16, 256)	1024
re_lu_1 (ReLU)	(None, 16, 16, 256)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 32, 32, 128)	524288
batch_normalization_1 (Batch Normalization)	(None, 32, 32, 128)	512
re_lu_2 (ReLU)	(None, 32, 32, 128)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 64, 64, 64)	131072
batch_normalization_2 (Batch Normalization)	(None, 64, 64, 64)	256
re_lu_3 (ReLU)	(None, 64, 64, 64)	0
conv2d (Conv2D)	(None, 64, 64, 3)	3075

```
=====  
Total params: 6,984,451  
Trainable params: 6,983,555  
Non-trainable params: 896
```



Model: "critic"

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 32, 32, 64)	3136
leaky_re_lu (LeakyReLU)	(None, 32, 32, 64)	0
conv2d_2 (Conv2D)	(None, 16, 16, 128)	131200
leaky_re_lu_1 (LeakyReLU)	(None, 16, 16, 128)	0
conv2d_3 (Conv2D)	(None, 8, 8, 128)	262272
leaky_re_lu_2 (LeakyReLU)	(None, 8, 8, 128)	0
flatten (Flatten)	(None, 8192)	0
dropout (Dropout)	(None, 8192)	0
dense_1 (Dense)	(None, 1)	8193

```
=====  
Total params: 404,801  
Trainable params: 404,801  
Non-trainable params: 0
```

No activation

https://colab.research.google.com/github/margaretmz/GANs-in-Art-and-Design/blob/main/4_wgan_anime_faces.ipynb

```

# UPDATE for WGAN: remove the code of real / fake labels
def train_step(self, real_images):

    batch_size = tf.shape(real_images)[0]
    noise = tf.random.normal(shape=(batch_size, self.latent_dim))

    # UPDATE for WGAN: we train the critic more often than the generator by 5 times (self.c_extra_steps)
    for i in range(self.c_extra_steps):
        # Step 1. Train the critic with both real images and fake images
        with tf.GradientTape() as tape:
            # Get the critic's predictions on the real images
            pred_real = self.critic(real_images, training=True)
            # Generate fake images from the latent noise
            fake_images = self.generator(noise, training=True)
            # Get the critic's predictions on the fake images
            pred_fake = self.critic(fake_images, training=True)
            # Calculate the critic loss with wasserstein loss function
            d_loss = self.d_loss_fn(pred_real, pred_fake)
            # Compute critic gradients
            grads = tape.gradient(d_loss, self.critic.trainable_variables)
            # Update critic weights
            self.d_optimizer.apply_gradients(zip(grads, self.critic.trainable_variables))

        # Step 2. Train the generator (do not update weights of the critic)
        with tf.GradientTape() as tape:
            fake_images = self.generator(noise, training=True)
            pred_fake = self.critic(fake_images, training=True)
            g_loss = self.g_loss_fn(pred_fake)
            # Compute generator gradients
            grads = tape.gradient(g_loss, self.generator.trainable_variables)
            # Update generator weights
            self.g_optimizer.apply_gradients(zip(grads, self.generator.trainable_variables))

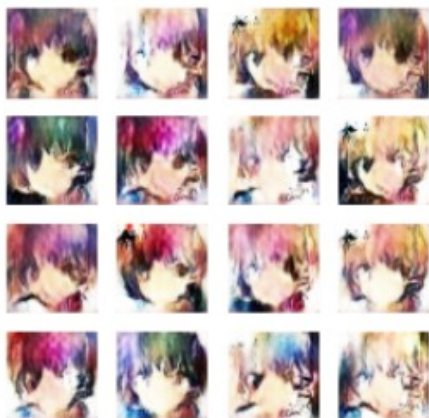
```

Epoch 1/50
249/249 [=====]



d_loss_fn()

Epoch 50/50
249/249 [=====]



g_loss_fn()

```

# Wasserstein loss for the critic
def d_wasserstein_loss(pred_real, pred_fake):
    real_loss = tf.reduce_mean(pred_real)
    fake_loss = tf.reduce_mean(pred_fake)
    return fake_loss - real_loss

# Wasserstein loss for the generator
def g_wasserstein_loss(pred_fake):
    return -tf.reduce_mean(pred_fake)

```

WGAN-GP: Wasserstein GANs with gradient penalty

The Wasserstein GAN with gradient penalty (WGAN-GP) introduces a gradient penalty term to the loss function.

This enforces the 1-Lipschitz constraint

Wasserstein Loss: Measures difference between discriminator scores on real vs. generated samples.

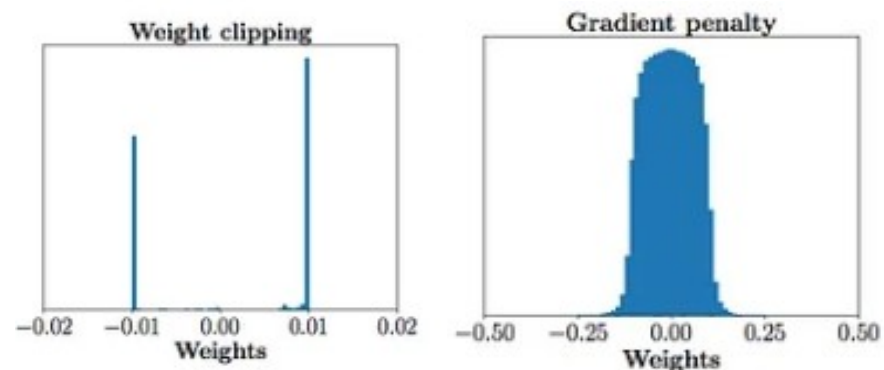
$$L = \mathbb{E}_{\tilde{x} \sim \mathbb{P}_g} [D(\tilde{x})] - \mathbb{E}_{x \sim \mathbb{P}_r} [D(x)]$$

Gradient Penalty (GP): Enforces 1-Lipschitz condition by penalizing deviation of gradient norm from 1.

$$GP = \lambda \mathbb{E}_{\hat{x} \sim \mathbb{P}_{\hat{x}}} [(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2]$$

Benefits of WGAN-GP:

- Provides smoother and more stable training dynamics.
- Achieves better convergence and reduces mode collapse.



WGAN-GP: Wasserstein GANs with gradient penalty

Algorithm 1 WGAN with gradient penalty. We use default values of $\lambda = 10$, $n_{\text{critic}} = 5$, $\alpha = 0.0001$, $\beta_1 = 0$, $\beta_2 = 0.9$.

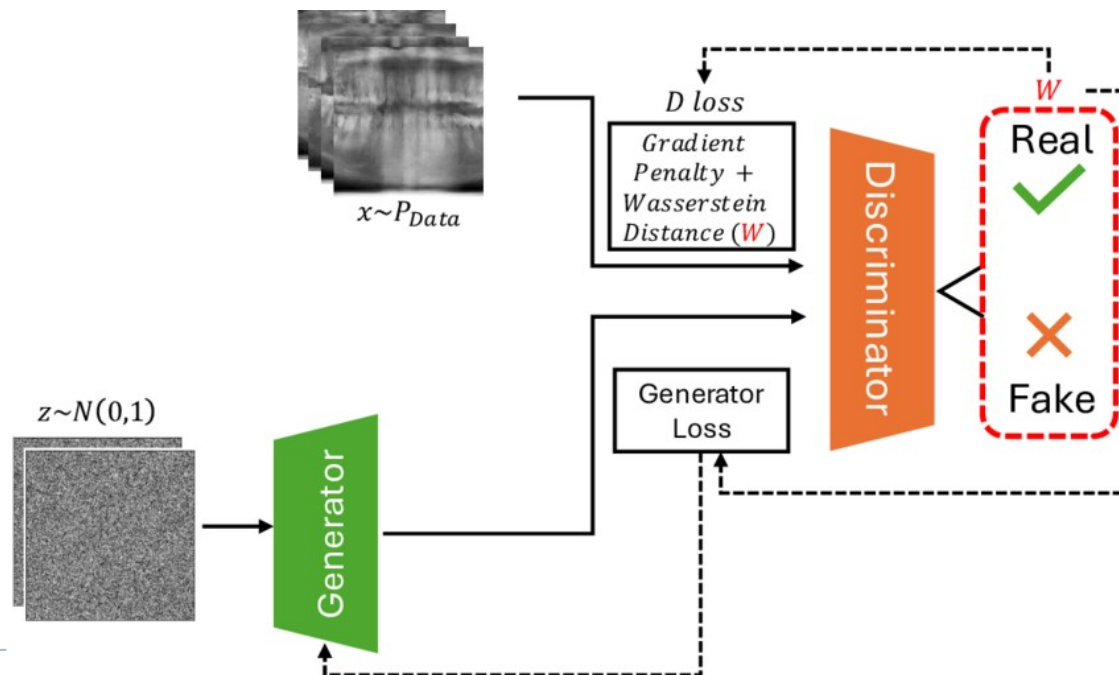
Require: The gradient penalty coefficient λ , the number of critic iterations per generator iteration n_{critic} , the batch size m , Adam hyperparameters α, β_1, β_2 .

Require: initial critic parameters w_0 , initial generator parameters θ_0 .

```
1: while  $\theta$  has not converged do
2:   for  $t = 1, \dots, n_{\text{critic}}$  do
3:     for  $i = 1, \dots, m$  do
4:       Sample real data  $x \sim \mathbb{P}_r$ , latent variable  $z \sim p(z)$ , a random number  $\epsilon \in U[0, 1]$ .
5:        $\tilde{x} \leftarrow G_\theta(z)$ 
6:        $\hat{x} \leftarrow \epsilon x + (1 - \epsilon)\tilde{x}$ 
7:        $L^{(i)} \leftarrow D_w(\tilde{x}) - D_w(x) + \lambda(\|\nabla_{\hat{x}} D_w(\hat{x})\|_2 - 1)^2$ 
8:     end for
9:      $w \leftarrow \text{Adam}(\nabla_w \frac{1}{m} \sum_{i=1}^m L^{(i)}, w, \alpha, \beta_1, \beta_2)$ 
10:   end for
11:   Sample a batch of latent variables  $\{z^{(i)}\}_{i=1}^m \sim p(z)$ .
12:    $\theta \leftarrow \text{Adam}(\nabla_\theta \frac{1}{m} \sum_{i=1}^m -D_w(G_\theta(z)), \theta, \alpha, \beta_1, \beta_2)$ 
13: end while
```

Training loop structure

1. Repeat n_{critic} times (e.g., 5):
 - Sample real data
 - Sample noise $\rightarrow$ generate fake data
 - Compute critic loss:
 - Wasserstein loss: $D(\text{fake}) - D(\text{real})$
 - Gradient penalty (enforces Lipschitz constraint)
 - Update critic
2. Once after critic updates:
 - Sample noise
 - Generate fake data
 - Compute generator loss: $-D(\text{fake})$
 - Update generator



UPDATE for WGAN: remove the code of real / fake labels

def train_step(self, real_images):

batch_size = tf.shape(real_images)[0]

noise = tf.random.normal(shape=(batch_size, self.latent_dim))

UPDATE for WGAN: we train the critic more often than the generator by 5 times (self.c_extra_steps)

for i in range(self.c_extra_steps):

Step 1. Train the critic with both real images and fake images

with tf.GradientTape() as tape:

Get the critic's predictions on the real images

pred_real = self.critic(real_images, training=True)

Generate fake images from the latent noise

fake_images = self.generator(noise, training=True)

Get the critic's predictions on the fake images

pred_fake = self.critic(fake_images, training=True)

Calculate the critic loss with wasserstein loss function

d_loss = self.d_loss_fn(pred_real, pred_fake)

Compute critic gradients

grads = tape.gradient(d_loss, self.critic.trainable_variables)

Update critic weights

self.d_optimizer.apply_gradients(zip(grads, self.critic.trainable_variables))

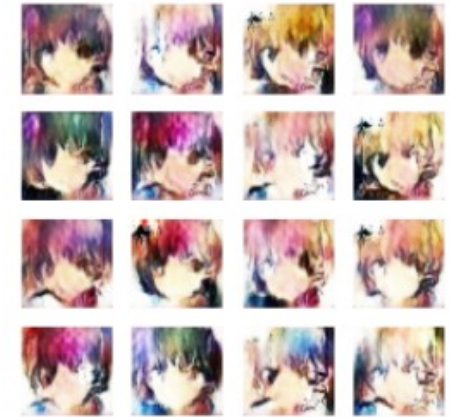
Step 2. Train the generator (do not update weights of the critic)

with tf.GradientTape() as tape:

WGAN

Epoch 50/50

249/249 [=====]



Epoch 50/50

249/249 [=====]



def train_step(self, real_images):

batch_size = tf.shape(real_images)[0]

noise = tf.random.normal(shape=(batch_size, self.latent_dim))

Train the critic more often than the generator by 5 times (self.c_extra_steps)

for i in range(self.c_extra_steps):

Step 1. Train the critic with both real images and fake images

with tf.GradientTape() as tape:

pred_real = self.critic(real_images, training=True)

fake_images = self.generator(noise, training=True)

pred_fake = self.critic(fake_images, training=True)

UPDATE for WGAN-GP: Calculate the gradient penalty

gp = self.gradient_penalty(batch_size, real_images, fake_images)

UPDATE for WGAN-GP: Add gradient penalty to the original critic loss

d_loss = self.d_loss_fn(pred_real, pred_fake) + gp * self.gp_weight

Compute critic gradients

grads = tape.gradient(d_loss, self.critic.trainable_variables)

Update critic weights

self.d_optimizer.apply_gradients(zip(grads, self.critic.trainable_variables))

WGAN-GP

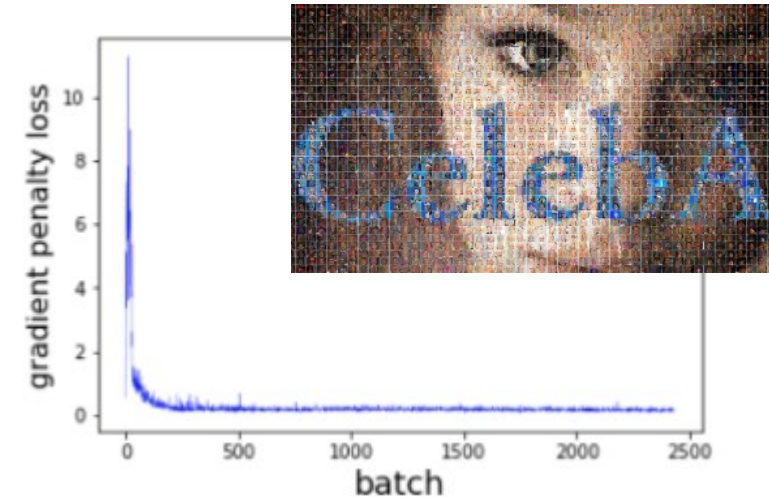
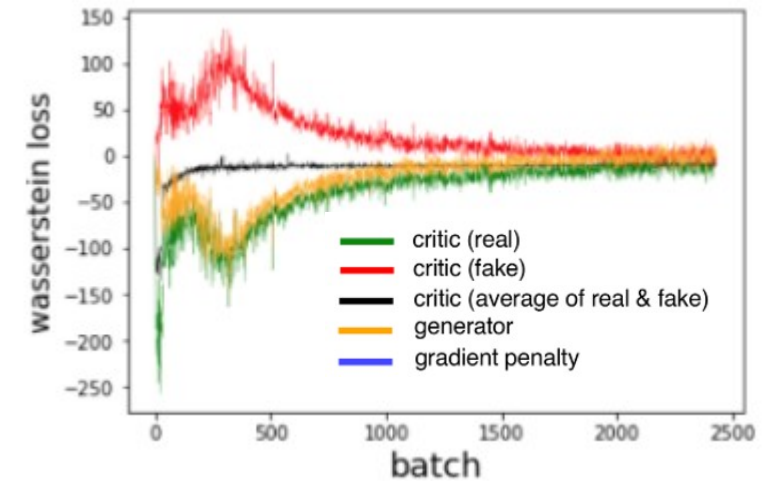
WGAN and WGAN-GP: other considerations

Two significant contributions for WGAN are

- **Reduce mode collapse**: the critic is more constructive than the discriminator
- GAN images are generally sharper than those from VAE, but longer and more complicated training.

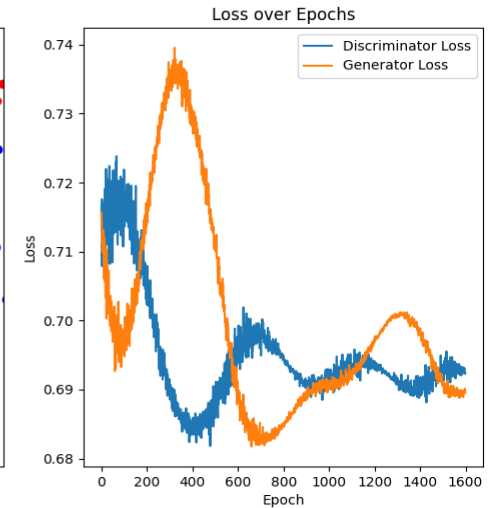
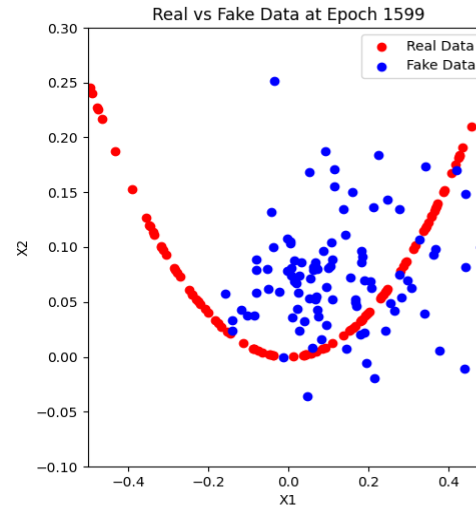
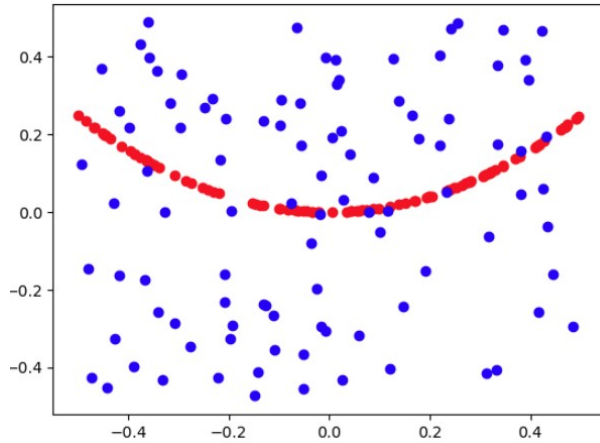


Figure 5: 1024×1024 images generated using the CELEBA-HQ dataset. See Appendix F for a larger set of results, and the accompanying video for latent space interpolations.



5C.6 Other GANs architectures

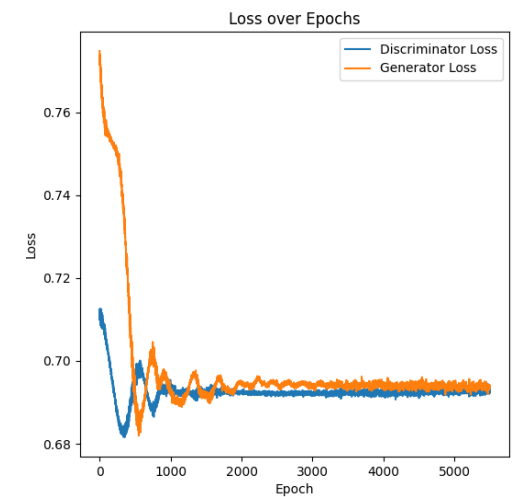
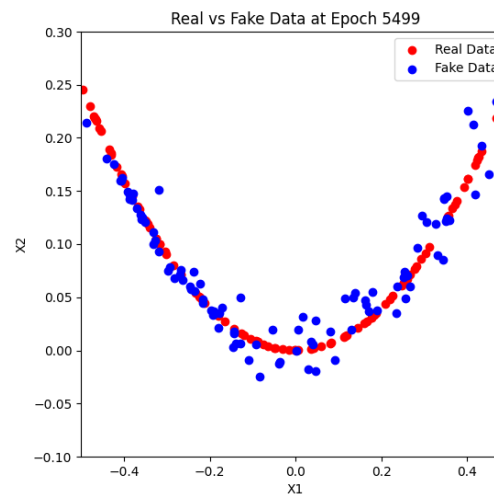
Progressive GANs



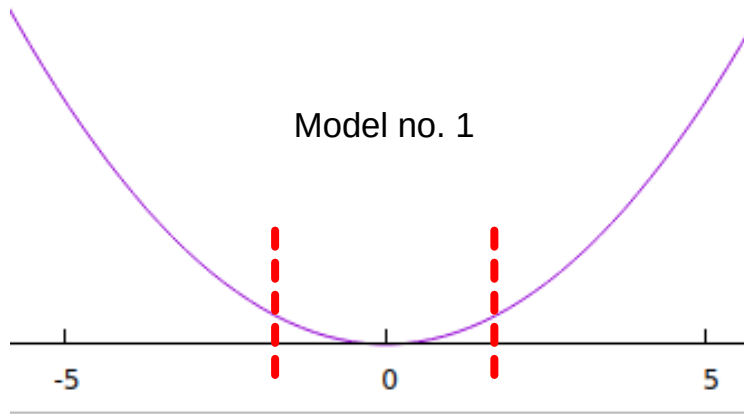
GANs are slow and difficult to train.

Is it possible to accelerate the training?

At least can we make it more robust?



Idea: train a GAN for a small domain and then expand it to a larger domain



```
def train(generator, discriminator, latent_dim, epochs=5000,
          batch_size=128, expand_rate=0.1, expansion_frequency=1000):
    criterion = nn.BCELoss()
    optimizer_G = optim.Adam(generator.parameters(), lr=0.0001, betas=(0.5, 0.999))
    optimizer_D = optim.Adam(discriminator.parameters(), lr=0.0001, betas=(0.5, 0.999))
    losses = []
    x_min, x_max = -0.2, 0.2 # Initial domain

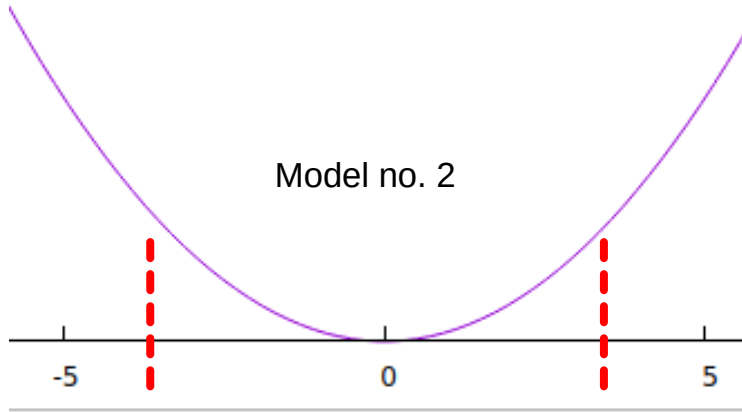
    plt.ion()
    fig, axs = plt.subplots(1, 2, figsize=(12, 6))

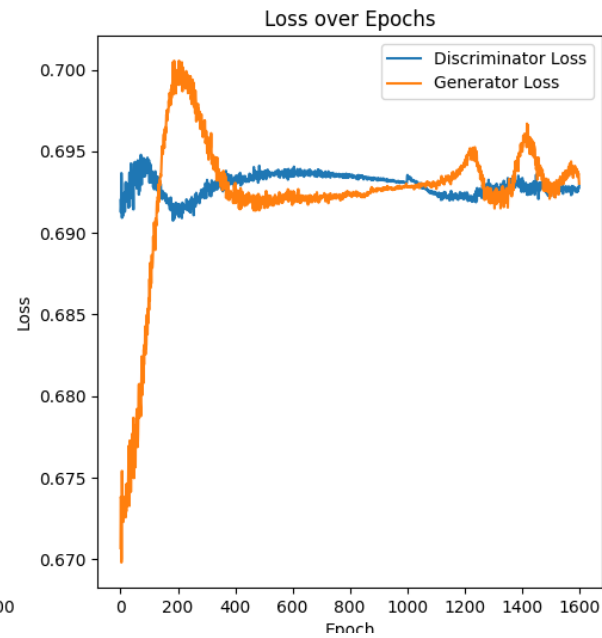
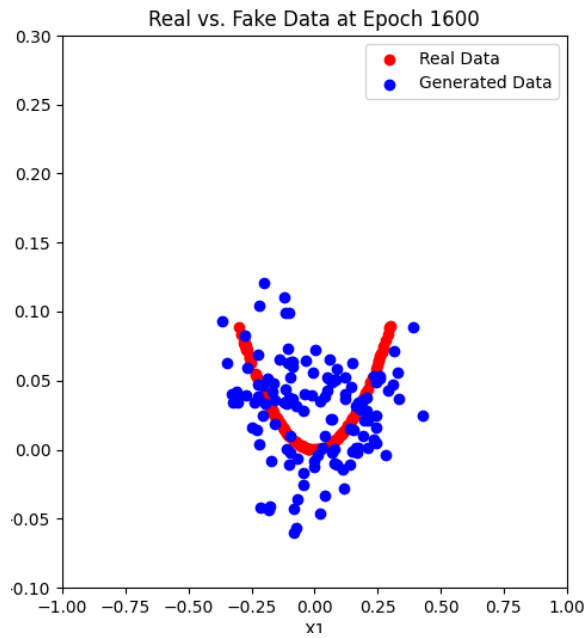
    for epoch in range(epochs):
        if epoch % expansion_frequency == 0 and epoch > 0:
            print("***EXPANDED***")
            x_min -= expand_rate # Expand domain
            x_max += expand_rate

        real_data, real_labels = generate_real_samples(batch_size, x_min, x_max)
        fake_data, fake_labels = generate_fake_samples(generator, latent_dim, batch_size)

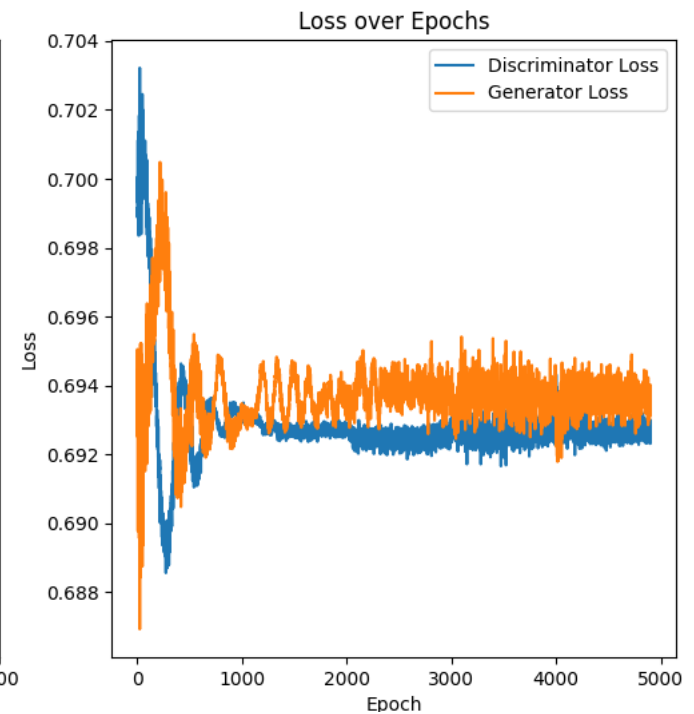
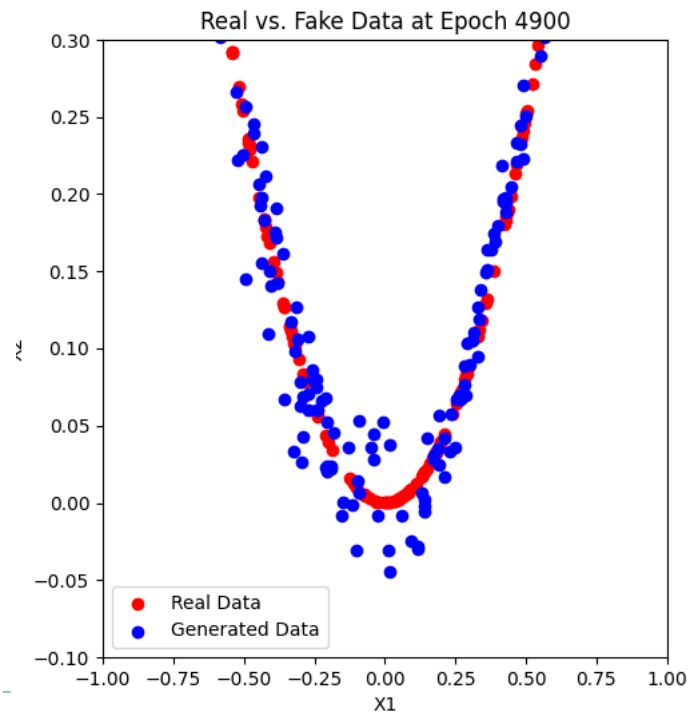
        # Train Discriminator
        optimizer_D.zero_grad()
        real_loss = criterion(discriminator(real_data), real_labels)
        fake_loss = criterion(discriminator(fake_data.detach()), fake_labels)
        d_loss = (real_loss + fake_loss) / 2
        d_loss.backward()
        optimizer_D.step()

        # Train Generator
        optimizer_G.zero_grad()
        g_loss = criterion(discriminator(fake_data), torch.ones(batch_size, 1))
        g_loss.backward()
        optimizer_G.step()
        losses.append((d_loss.item(), g_loss.item()))
```

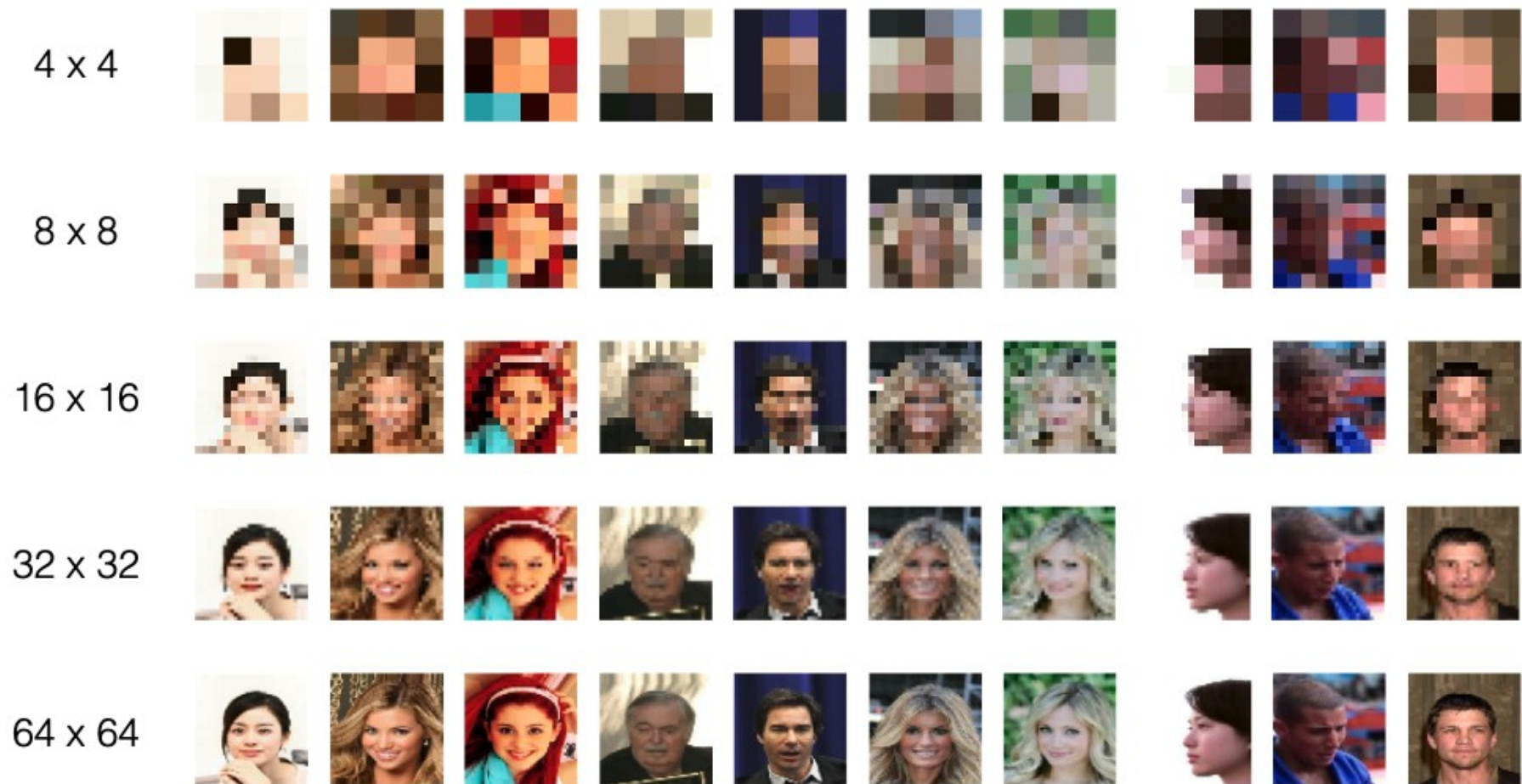




Results of the simple model



Progressive training



Developed by NVidia Labs to improve the speed and stability of GAN training

https://research.nvidia.com/publication/2018-04_progressive-growing-gans-improved-quality-stability-and-variation



Results



CelebA dataset

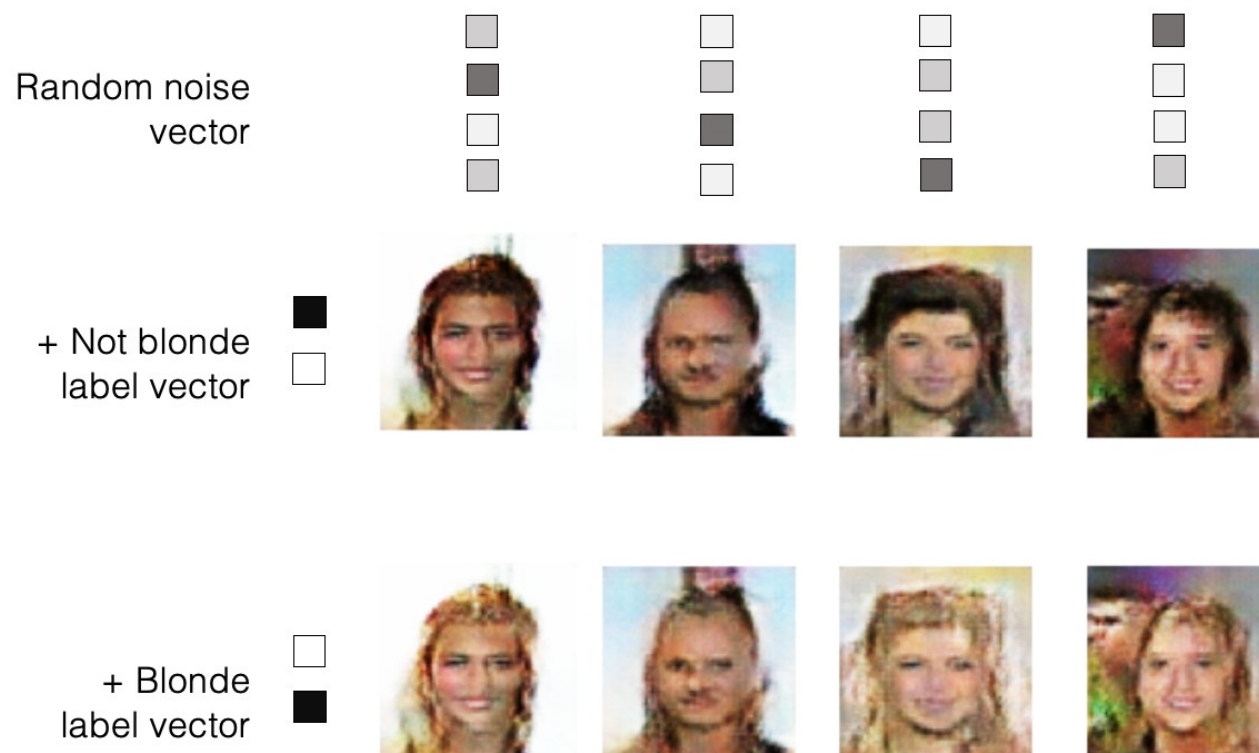


LSUN dataset

Conditional GANs

Concretely, the conditional GAN game is just the GAN game with class labels provided:

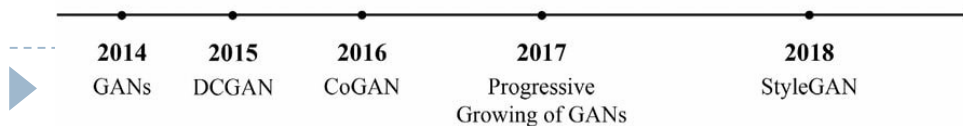
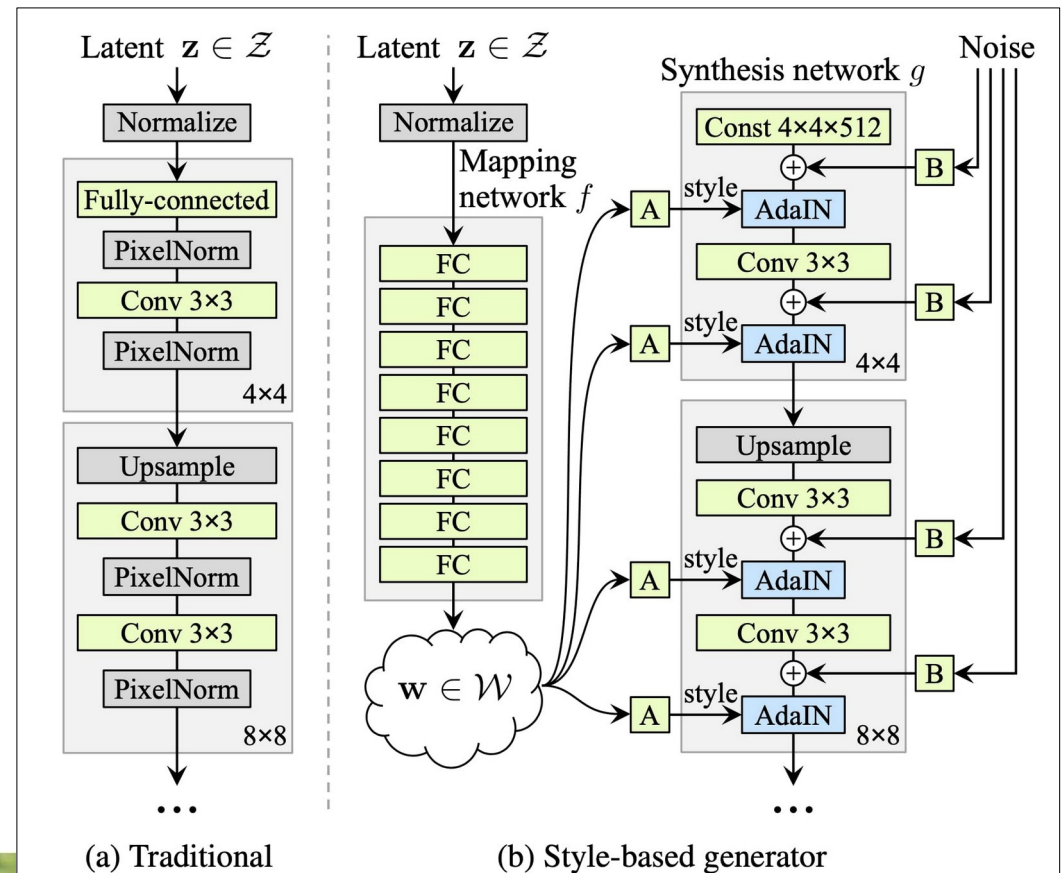
$$L(\mu_G, D) := \mathbb{E}_{c \sim \mu_C, x \sim \mu_{ref}(c)} [\ln D(x, c)] + \mathbb{E}_{c \sim \mu_C, x \sim \mu_G(c)} [\ln(1 - D(x, c))]$$



StyleGANs

StyleGANs introduce a style-based generator, enabling precise control over image attributes through manipulated 'style vectors' derived from a learned latent space.

- In a disentangled latent space ($\mathcal{W}$), changing one component of the latent vector affects only the specific aspect of the generated image associated with that component, without altering other features.
- Enables control of features: high-level (e.g., face orientation) and low-level (e.g., hair placement) features controlled independently



There are two probability spaces (Ω_X, μ_X) , (Ω_Y, μ_Y) , corresponding to the two domains needed for translations fore-and-back.

There are 4 players in 2 teams: generators $G_X : \Omega_X \rightarrow \Omega_Y$, $G_Y : \Omega_Y \rightarrow \Omega_X$, and discriminators $D_X : \Omega_X \rightarrow [0, 1]$, $D_Y : \Omega_Y \rightarrow [0, 1]$.

The objective function is

$$L(G_X, G_Y, D_X, D_Y) = L_{GAN}(G_X, D_X) + L_{GAN}(G_Y, D_Y) + \lambda L_{cycle}(G_X, G_Y)$$

where λ is a positive adjustable parameter, L_{GAN} is the GAN game objective, and L_{cycle} is the *cycle consistency loss*:

$$L_{cycle}(G_X, G_Y) = E_{x \sim \mu_X} \|G_X(G_Y(x)) - x\| + E_{y \sim \mu_Y} \|G_Y(G_X(y)) - y\|$$

The generators aim to minimize the objective, and the discriminators aim to maximize it:

$$\min_{G_X, G_Y} \max_{D_X, D_Y} L(G_X, G_Y, D_X, D_Y)$$



CycleGAN is an architecture for performing translations between two domains, such as between photos of horses and photos of zebras, or photos of night cities and photos of day cities.

Imagine you want to turn photos of horses into zebras, but you don't have matching pairs (the same scene with both a horse and a zebra).

A CycleGAN solves this by training two generators:

- One maps from domain A $\rightarrow$ B (horse $\rightarrow$ zebra)
- Another maps from B $\rightarrow$ A (zebra $\rightarrow$ horse)

It also uses two discriminators to judge whether the generated images look real in each domain.

The key innovation is cycle consistency:

- If you translate A $\rightarrow$ B and then back B $\rightarrow$ A, you should recover the original image.
- This constraint prevents the model from making arbitrary changes and forces it to preserve structure.

Inception score

Steps:

1. KL divergence:

$$\text{KL}(p(y|x) \parallel p(y)) = \sum_y p(y|x) \cdot \log \frac{p(y|x)}{p(y)}$$

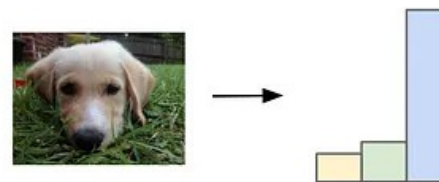
2. Exponentiate:

$$\text{IS} = \exp(\mathbb{E}_{x \sim p_g} [\text{KL}(p(y|x) \parallel p(y))])$$

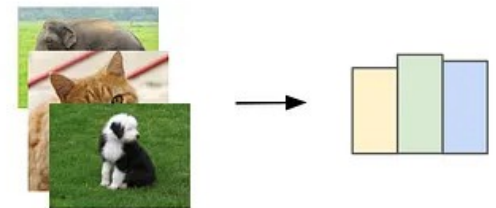
- ▶ **Objective:** Quantifying the quality of generated samples in Generative Adversarial Networks (GANs)
- ▶ Inception Score (IS) measures two aspects:
 1. Diversity of generated samples
 2. Fidelity to the real data distribution
- ▶ **Process:**
 1. Compute the KL divergence between the real and generated distributions
 2. Exponentiate the result of step 1

Intuitively, if we have a GAN that generates images of animals, we want each image to be classified clearly as a type of animal, but we want diversity of animals.

Then the GAN will create content diverse, but each image is well generated



Ideal label distribution



Ideal marginal distribution

► **Intuition:**

- A higher IS indicates better quality in generated samples
- Encourages diversity by comparing $p(y|x)$ to $p(y)$
- Fidelity is ensured by measuring KL divergence

Image	Category	Score
	daisy	0.05646
	book jacket, dust cover, dust jacket, dust wrapper	0.05086
	goldfish, Carassius auratus	0.04913
	hummingbird	0.02358
	panpipe, pandean pipe, syrinx	0.02029

Fréchet inception distance (FID)

- ▶ **Objective:** Measure the similarity between generated samples and real data in Generative Adversarial Networks (GANs)
- ▶ The high-level idea is to find a generated distribution of samples that minimizes the number of modifications required to make the generated distribution look like the distribution of the true data
- ▶ FID is calculated by running images through an Inception network and comparing intermediate representations (feature maps or layers) rather than the final output
- ▶ FID evaluates the distance of the embedded means, variances, and covariances of the two distributions (real and generated)

- ▶ Given two multivariate Gaussian distributions, $P_r \sim \mathcal{N}(\mu_r, \Sigma_r)$ (real data) and $P_g \sim \mathcal{N}(\mu_g, \Sigma_g)$ (generated data), the FID is defined as:

$$\text{FID}(P_r, P_g) = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

- ▶ To calculate FID, follow these steps:
 1. Pass both real and generated images through the Inception network to obtain their intermediate representations
 2. Calculate the mean and covariance of the intermediate representations for real and generated images
 3. Compute the FID using the formula above
- ▶ A lower FID indicates that the generated samples are more similar to the real data distribution, while a higher FID implies that the generated samples deviate significantly from the real data distribution

```

# prepare the inception v3 model
model = InceptionV3(include_top=False, pooling='avg', input_shape=(299,299,3))

# load cifar10 images
(images1, _), (images2, _) = cifar10.load_data()
shuffle(images1)
images1 = images1[:10000]
print('Loaded', images1.shape, images2.shape)
# convert integer to floating point values
images1 = images1.astype('float32')
images2 = images2.astype('float32')
# resize images
images1 = scale_images(images1, (299,299,3))
images2 = scale_images(images2, (299,299,3))
print('Scaled', images1.shape, images2.shape)
# pre-process images
images1 = preprocess_input(images1)
images2 = preprocess_input(images2)
# calculate fid
fid = calculate_fid(model, images1, images2)
print('FID: %.3f' % fid)

```

```

# calculate frechet inception distance
def calculate_fid(model, images1, images2):
    # calculate activations
    act1 = model.predict(images1)
    act2 = model.predict(images2)
    # calculate mean and covariance statistics
    mu1, sigma1 = act1.mean(axis=0), cov(act1, rowvar=False)
    mu2, sigma2 = act2.mean(axis=0), cov(act2, rowvar=False)
    # calculate sum squared difference between means
    ssdiff = numpy.sum((mu1 - mu2)**2.0)
    # calculate sqrt of product between cov
    covmean = sqrtm(sigma1.dot(sigma2))
    # check and correct imaginary numbers from sqrt
    if iscomplexobj(covmean):
        covmean = covmean.real
    # calculate score
    fid = ssdiff + trace(sigma1 + sigma2 - 2.0 * covmean)
    return fid

```